

claude-windows / 96963dab-d38f-488c-af44-530ddbfeed1

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf_00bebd3c-82f\agent-a43fa76865aa021a7.jsonl`  
- SHA-256: `24913b1ecd32c67e7ec76da6c0c00b4dfb6e8ee989ea203754e7a57bff52eba9`  
- Source modified: `2026-07-08T05:52:13+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_00bebd3c-82f`  
- session_id: `96963dab-d38f-488c-af44-530ddbfeed1`
```

Transcript

1. user (2026-07-08T05:44:49.636Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything).
Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.
Subsystem: ORCHESTRATION PROCESSING PATH. Map backend/orchestration.py's _execute_processing from job pickup to completion: the storage.acquire_local_input call (~line 748), the compute_video_id re-hash (~line 751), how/when validation runs (which function, which validators), the _maybe_dispatch_remote / compute_target decision for detection, what payload goes to the remote worker, how results are ingested back (play_segments/DB writes), and cleanup of scratch files. Also map _execute_compile + _resolve_compile ("run at SUBMIT AND in the WORKER") and the _LOCAL_COMPUTE_LANE serialization. Note exactly which steps would change if (1) video_id came from the request instead of the re-hash, and (2) validation ran on the worker instead of the CP.

2. user (2026-07-08T05:44:57.254Z)

```
1204 backend/orchestration.py  
490 backend/api/job_worker.py  
33 backend/utils/hashing.py  
1727 total  
__init__.py  
__pycache__  
base.py  
cloud_run.py
```

3. user (2026-07-08T05:45:03.645Z)

```
1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).  
2  
3      The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the  
4      Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /  
``_ingest_remote_segments``),  
5      and the shared compile resolution + worker handler (``_resolve_compile`` /  
``_execute_compile``).  
6      Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is  
unchanged.  
7  
8      SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must  
remain
```

```

9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import ValidationResult, registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and

```

```

61     keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62     """
63     if segments:
64         db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65     else:
66         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))
75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
scratch."""
82         storage_config = (
83             config.get("storage")
84             if isinstance(config, dict)
85             else getattr(config, "storage", None)
86         )
87         try:
88             if storage.input_is_local(raw_input, storage_config):
89                 return config
90         except ValueError:
91             return config
92         try:
93             cfg = (
94                 config.model_dump(mode="json")
95                 if hasattr(config, "model_dump")
96                 else dict(config or {})
97             )
98         except Exception:
99             cfg = {}
100         report = dict(cfg.get("report") or {})
101         if report.get("output_dir"):
102             return config
103         report["output_dir"] = os.path.join(output_base(config), "reports")
104         cfg["report"] = report
105         return cfg
106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
112         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
113         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
114         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
115         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
116         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
117         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as

```

```

a job failure
118     and prunes any partial rows (destroy-AFTER-confirm)."""
119     uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
120     ref = storage.parse_uri(uri)
121     n = 0
122     with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123         local = storage.fetch(
124             uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125         )
126         with open(local, newline="", encoding="utf-8") as f:
127             for row in csv.DictReader(f):
128                 if n >= max_rows:
129                     logger.warning(
130                         "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                         max_rows,
132                         uri,
133                     )
134                     break
135                 try:
136                     start, end = float(row["start"]), float(row["end"])
137                 except (KeyError, TypeError, ValueError):
138                     continue
139                 if not (math.isfinite(start) and math.isfinite(end)):
140                     continue # reject inf/nan times rather than persist a junk rally
141                 meta: Dict[str, Any] = {
142                     "source": "cloud_run",
143                     "ending_reason": (row.get("ending_reason") or "").strip(),
144                 }
145                 sc = (row.get("shot_count") or "").strip()
146                 if sc:
147                     try:
148                         meta["shot_count"] = int(
149                             float(sc)
150                         ) # OverflowError on 'inf'/'1e400'
151                     except (ValueError, OverflowError):
152                         meta["shot_count"] = sc
153                 db.add_play_segment(video_id, start, end, metadata=meta)
154                 n += 1
155     return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).

```

```

171     A non-weights segmenter returns True (nothing local to check)."""
172     name = (seg_name or "").strip().lower()
173     if name not in _WEIGHTS_BACKED_SEGMENTERS:
174         return True
175     idx = getattr(cfg, "indexing", None)
176     substrate = (
177         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178     ).strip().lower()
179     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181     else: # wasb_hybrid, or fusion on the (default) WASB substrate
182         path = (
183             os.environ.get("WASB_WEIGHTS")
184             or getattr(getattr(idx, "wasb", None), "weights_path", "")
185             or ""
186         )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190 def _cloud_available(cfg) -> bool:
191     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193     True when the control plane is wired for Cloud Run: either the configured default
target is
194     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195     (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196     are present so a forced per-request override can actually dispatch + collect a
result.
197
198     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199     deployment = getattr(cfg, "deployment", None)
200     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201         return True
202     has_job = bool(
203         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204     )
205     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206     return bool(has_job and out_root)
207
208
209 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213     (queue/cold-start), so the very first tick already shows movement."""
214     minutes = int(elapsed_sec) // 60
215     if minutes < 1:
216         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no

```

```

224     ``md5Hash``. Never raises ? a metadata miss must not block a submit.""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None
227     sc = _storage_settings(cfg)
228     if not sc:
229         return None
230     try:
231         st = storage.get_storage("gcs", config=sc).stat(input_ref)
232         return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
246         for
247         both producers (the drain's DB-cache short-circuit and the submit-time cache,
248         #484)."""
249         return {
250             "video_id": video_id,
251             "status": "success",
252             "message": (
253                 f"Reused {len(segments)} cached play segments. "
254                 "Pass force=true to reprocess this video."
255             ),
256             "results": validation_results or [],
257             "play_segments": segments,
258             "cached": True,
259             "compute": {"path": "cached", "requested": requested_compute},
260         }
261
262     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
263         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
264         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
265         decide."""
266         ref = storage.parse_uri(uri)
267         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
268             local = storage.fetch(
269                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
270             )
271             with open(local, encoding="utf-8") as f:
272                 loaded = json.load(f)
273             return loaded if isinstance(loaded, dict) else None
274
275     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
276     hits
277     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
278     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
279     instance
280     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
281     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
282     _REHYDRATE_LOCKS_GUARD = threading.Lock()
283
284     def _rehydrate_lock(video_id: str) -> threading.Lock:
285         with _REHYDRATE_LOCKS_GUARD:

```

```

284         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
285
286
287     def probe_process_cache(db, cfg, video_id: str) -> bool:
288         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293         errors report False (? the normal dispatch path with its guards)."""
294         try:
295             existing = db.get_video(video_id)
296             if (
297                 existing
298                 and existing.get("status") == "processed"
299                 and db.query_play_segments(video_id, {})
300             ):
301                 return True
302             sc = _storage_settings(cfg)
303             if not sc or (sc.get("backend") or "local") == "local":
304                 return False
305             output_root = (sc.get("output_root") or "").rstrip("/")
306             if not output_root:
307                 return False
308             report_uri = f"{output_root}/outputs/{video_id}/report.json"
309             if not storage.exists(report_uri, config=sc):
310                 return False
311             report = _read_bucket_json(report_uri, sc)
312             return bool(report and report.get("status") == "success")
313         except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314             logger.warning(
315                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316             )
317         return False
318
319
320     def cached_process_result(
321         db, cfg, video_id: str, video_path: str, requested_compute=None
322     ) -> Optional[Dict[str, Any]]:
323         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
325         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
326         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
327         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
329         partial earlier ingest is replaced, never double-counted.
330
331         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
333         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
334         job before invoking (writes happen post-claim)."""
335         try:
336             with _rehydrate_lock(video_id):
337                 existing = db.get_video(video_id)
338                 if existing and existing.get("status") == "processed":

```

```

339         segs = db.query_play_segments(video_id, {})
340     if segs:
341         return _cached_process_payload(
342             video_id,
343             existing.get("validation_results", []),
344             segs,
345             requested_compute,
346         )
347     sc = _storage_settings(cfg)
348     if not sc or (sc.get("backend") or "local") == "local":
349         return None
350     output_root = (sc.get("output_root") or "").rstrip("/")
351     if not output_root:
352         return None
353     outputs_uri = f"{output_root}/outputs/{video_id}"
354     report_uri = f"{outputs_uri}/report.json"
355     if not storage.exists(report_uri, config=sc):
356         return None
357     report = _read_bucket_json(report_uri, sc)
358     if not report or report.get("status") != "success":
359         return None
360     if not video_path:
361         # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
362         # carries the worker's --video-uri (P3); older reports fall back to the
363         # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
364         # recover, and the next same-md5 submit re-binds the truthful path via
365         # add_video's upsert.
366         video_path = str(report.get("video_uri") or "") or _first_object_under(
367             f"{output_root}/videos/{video_id}/", sc
368         )
369     play_wm, cand_wm = db.segment_watermarks(video_id)
370     db.add_video(video_id, video_path, "processed", [])
371     n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
372     if n <= 0:
373         return None
374     segments = [
375         s
376         for s in db.query_play_segments(video_id, {})
377         if int(s.get("id", 0)) > play_wm
378     ]
379     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
380     if not segments:
381         return None
382     logger.info(
383         "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
384         n,
385         video_id,
386         outputs_uri,
387     )
388     return _cached_process_payload(video_id, [], segments, requested_compute)
389 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
390     logger.warning(
391         "[API] submit-time cache check failed for %s ? dispatching normally",
392         video_id,
393         exc_info=True,
394     )
395     return None
396
397
398 def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399     """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400     try:

```



```

401         return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402     except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403         return ""
404
405
406     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
412         so
413         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
414         DB
415         has the video's rows now (re-query). Non-md5 ids return False without touching the
416         bucket; every failure degrades to False (the read just reports what the DB has)."""
417         vid = (video_id or "").strip().lower()
418         if not _MD5_HEX_RE.fullmatch(vid):
419             return False
420         if not probe_process_cache(db, cfg, vid):
421             return False
422         return cached_process_result(db, cfg, vid, "") is not None
423
424     def _maybe_dispatch_remote(
425         cfg,
426         db,
427         req: "ProcessRequest",
428         video_id: str,
429         seg_name: str,
430         val_results,
431         play_wm: int,
432         cand_wm: int,
433         notify,
434     ) -> Optional[Tuple[Dict[str, Any], bool]]:
435         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
436         ``deployment.compute_target
437         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
438         intervals into
439         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
440
441         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
442         the cloud job
443         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
444         observability report
445         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
446         process segmenter
447         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
448         non-``cloud_run`` target, so
449         the in-process path stays byte-identical)."""
450         from backend.compute import ComputeJobSpec, get_compute_backend
451
452         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
453             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
454             (id > play_wm);
455             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
456             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
457             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
458             only when the
459             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
460             rc / ingest
461             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
462             records the
463             # intended backend, `requested` the picker choice, `dispatched` whether the
464             remote job ran.

```

```

453         return (
454             {
455                 "video_id": video_id,
456                 "status": "failed",
457                 "message": msg,
458                 "results": [r.model_dump() for r in val_results],
459                 "play_segments": [],
460                 "compute": {
461                     "path": "cloud_run" if ran else "not_run",
462                     "requested": req.compute,
463                     "target": "cloud_run",
464                     "dispatched": ran,
465                 },
466             },
467             ran,
468         )
469
470     # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472     # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473     choice = (req.compute or "").strip().lower()
474     if choice and choice not in ("local", "cloud"):
475         logger.warning(
476             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477             req.compute,
478         )
479     choice = ""
480     if choice == "local":
481         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487         # below exactly as before (the weights-present check is the escape hatch).
488         if (
489             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490             and _cloud_available(cfg)
491             and not _local_detector_weights_present(cfg, seg_name)
492         ):
493             return _failed(
494                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
496                 False,
497             )
498     return None # explicit "run on my machine" ? in-process regardless of the
server's target
499     if choice == "cloud":
500         if not _cloud_available(cfg):
501             return _failed(
502                 "cloud-serving was requested but this server isn't configured for it "
503                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",

```

```

504         False,
505     )
506     compute = get_compute_backend(cfg, target_override="cloud_run")
507 else:
508     compute = get_compute_backend(
509         cfg
510     ) # server default (default-OFF unless cloud_run)
511 if compute is None:
512     return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514 storage_cfg = getattr(cfg, "storage", None)
515
516 # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517 try:
518     input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519 except ValueError as e:
520     return _failed(
521         f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522     )
523 if input_ref.backend == "local":
524     # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525     # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526     # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527     # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528     # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529     # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530     # default, while a bare/local-looking root can't stage no matter what
input_backend says
531     # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532     # gives us nothing to build the staged URI under).
533     input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534     try:
535         root_backend = (
536             storage.parse_uri(input_root).backend if input_root else "local"
537         )
538     except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539         root_backend = "local"
540     if root_backend == "local":
541         return _failed(
542             "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543             "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544             "Run job.",
545             False,
546         )
547     local_src = input_ref.key # resolved local path of the upload
548     staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549     try:
550         notify("staging upload to bucket")
551         # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553         storage.put(

```

```

554         local_src,
555         staged_uri,
556         config=(
557             storage_cfg.model_dump() if storage_cfg is not None else None
558         ),
559     )
560     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
561         logger.error(
562             "[API] cloud-serving: failed to stage %s -> %s: %s",
563             local_src,
564             staged_uri,
565             e,
566             exc_info=True,
567         )
568         return _failed(
569             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570             False,
571         )
572     # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573     # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574     # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575     req.video_path = staged_uri
576     input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577     db.add_video(
578         video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579     )
580     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581     if not out_root:
582         return _failed(
583             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584             "? that's where the Cloud Run job writes the result.",
585             False,
586         )
587
588     notify("dispatching (cloud_run)")
589     logger.info(
590         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591         req.video_path,
592         out_root,
593     )
594     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model

```

```

606         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)
610         if wasb_cfg is not None:
611             decode_backend = getattr(wasb_cfg, "decode_backend", None)
612             if decode_backend:
613                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614             batch_size = getattr(wasb_cfg, "batch_size", None)
615             if batch_size:
616                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
617             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618             if prefetch:
619                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
621         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
622         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
623         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
624         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
625         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
626         spec = ComputeJobSpec(
627             video_uri=req.video_path,
628             out_uri=out_root,
629             segmenter=seg_name,
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,
640                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641             )
642         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
643             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644             return _failed(f"cloud dispatch error: {e}", True)
645         if result.returncode != 0:
646             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
647         if result.video_id and result.video_id != video_id:
648             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
650             logger.warning(
651                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
652                 "and the worker hash; ingesting under the local id.",
653                 result.video_id,
654                 video_id,
655             )
656

```

```

657         notify("ingesting (cloud_run)")
658         storage_dict = (
659             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
660             if hasattr(storage_cfg, "model_dump")
661             else {}
662         )
663         try:
664             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
665         except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
666             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
667             return _failed(
668                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
669                 True,
670             )
671
672         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
673         segments = [
674             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
675         ]
676         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
677         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
678         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679         provenance = {
680             "path": "cloud_run",
681             "requested": req.compute,
682             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
684             "execution": result.execution,
685             "outputs_uri": result.outputs_uri,
686         }
687         logger.info(
688             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
689             result.execution,
690             provenance["job"],
691             provenance["region"],
692             result.outputs_uri,
693             video_id,
694         )
695         return (
696             {
697                 "video_id": video_id,
698                 "status": "success",
699                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
700                 "results": [r.model_dump() for r in val_results],
701                 "play_segments": segments,
702                 "compute": provenance,
703             },
704             True,
705         )
706
707
708     def _execute_processing(
709         config=None, db=None, req=None, progress=None
710     ) -> Dict[str, Any]:
711         """The full hash ? validate ? segment ? report pipeline for one video.
712
713         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714         module-level ``global_config``/``db_instance`` (backward-compatible with existing

```

```

715     tests that pass only ``req``). Route handlers always pass explicit params; older
716     test code that calls ``_execute_processing(req)`` still works.
717
718     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
719     Raises on unexpected internal errors ? the caller records those as a job failure
720     """
721     import backend.main as _main # call-time seam: main's globals/model stay patchable
722
723     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
724     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
725     if config is not None and not isinstance(config, AppConfig):
726         # Old signature: _execute_processing(req, progress=None)
727         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
728             req = db
729             db = None
730         elif req is None and isinstance(config, _main.ProcessRequest):
731             req = config
732             config = None
733     # Resolve config/db from module globals when not passed explicitly
734     if config is None:
735         config = _main.global_config
736     if db is None:
737         db = _main.db_instance
738
739     notify = progress or (lambda stage: None)
740
741     notify("hashing")
742     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
743     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
744     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
745     # consumers (hash / validators / segmenter) use the resolved local path.
746     local_video = ""
747     try:
748         local_video = storage.acquire_local_input(
749             req.video_path, getattr(config, "storage", None)
750         )
751         video_id = _main.compute_video_id(local_video)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     try:
758         existing_video = db.get_video(video_id)
759     except Exception:
760         storage.cleanup_acquired_local_input(
761             req.video_path, local_video, getattr(config, "storage", None)
762         )
763         raise
764     was_reingest = existing_video is not None
765
766     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
767     default_seg = getattr(
768         getattr(config, "indexing", None), "default_segmenter", "motion"
769     )
770     seg_name = req.segmenter or default_seg
771     segmenter_actual = None
772     segmentation_ran = False
773     val_results: List[Any] = []
774     val_context: Dict[str, Any] = {}

```

```

775         # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
776         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
777         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
778         compute_actual: Optional[str] = None
779         response: Optional[Dict[str, Any]] = None
780         try:
781             if (
782                 existing_video
783                 and existing_video.get("status") == "processed"
784                 and not req.force
785             ):
786                 cached_segments = db.query_play_segments(video_id, {})
787                 if cached_segments:
788                     logger.info(
789                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                         video_id,
791                     )
792                 response = _cached_process_payload(
793                     video_id,
794                     existing_video.get("validation_results", []),
795                     cached_segments,
796                     getattr(req, "compute", None),
797                 )
798                 return response
799
800         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801         #     UNLESS this exact content already has a cached verdict under the current
validator config.
802         notify("validating")
803         # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804         # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805         # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806         # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807         # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808         # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809         # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810         # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812         # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813         # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814         val_fingerprint = registry.config_fingerprint(config)
815         # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817         # failing the job. The cache is a pure speedup ? never a new failure mode.
818         reused_validation = False
819         if not req.force:
820             try:

```



```

821         cached = db.get_validation_cache(video_id)
822         if cached is not None and cached.get("fingerprint") == val_fingerprint:
823             # Replay the stored ValidationResult list; val_context stays {} (no
fresh
824             # ffmpeg_meta ? the report's media block degrades gracefully to
size-only, the
825             # deliberate cost of skipping the decode). Segmentation reads
neither of these.
826             val_results = [
827                 ValidationResult(**r) for r in cached.get("results", [])
828             ]
829             reused_validation = True
830             logger.info(
831                 "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
832                 "validator config unchanged ? skipping the validator suite. "
833                 "force=true re-validates.",
834                 cached.get("passed"),
835                 video_id,
836             )
837         except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
838             reused_validation = False
839             logger.warning(
840                 "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
841                 video_id,
842                 exc_info=True,
843             )
844         if not reused_validation:
845             logger.info(f"Running validations for {req.video_path}...")
846             val_results, val_context = registry.run_all_with_context(local_video,
config)
847             failures = [r for r in val_results if not r.passed]
848             if not reused_validation:
849                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
850                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
851                 db.set_validation_cache(
852                     video_id,
853                     val_fingerprint,
854                     not failures,
855                     [r.model_dump() for r in val_results],
856                 )
857
858             if failures:
859                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
860                 # status; it no longer destroys the video's prior good segments.
861                 db.add_video(
862                     video_id,
863                     req.video_path,
864                     "failed",
865                     [r.model_dump() for r in val_results],
866                 )
867             response = {
868                 "video_id": video_id,
869                 "status": "failed",
870                 "message": _validation_failure_message(failures),
871                 "results": [r.model_dump() for r in val_results],
872             }
873             return response
874
875         # 2. Run segmenter & indexer
876         logger.info("[API] Video passed checks. Segmenting and indexing...")

```

```

877         db.add_video(
878             video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
879         )
880
881     segmenter_cls = segmenter_registry.get(seg_name)
882     if not segmenter_cls:
883         # Submit-time validation already 400s unknown names; this is the defensive
884         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
885         available = list(segmenter_registry.list_available().keys())
886         response = {
887             "video_id": video_id,
888             "status": "failed",
889             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
890             "results": [r.model_dump() for r in val_results],
891             "play_segments": [],
892         }
893         return response
894
895     logger.info(f"[API] Using segmenter: {seg_name}")
896     notify(f"segmenting ({seg_name})")
897     segmenter_actual = seg_name
898     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
899     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
900     play_wm, cand_wm = db.segment_watermarks(video_id)
901     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
902     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
903     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
904     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
905     # they're dropped on the cloud path in this slice ? a documented follow-up.)
906     remote = _maybe_dispatch_remote(
907         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
908     )
909     if remote is not None:
910         response, segmentation_ran = (
911             remote # `ran` = did the cloud job actually execute (report)
912         )
913         return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
914     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
915     compute_actual = "in_process"
916     segmenter = segmenter_cls(db, config)
917     try:
918         # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
919         # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
920         with _LOCAL_COMPUTE_LANE:
921             result = segmenter.process_video(
922                 video_id, local_video, req.player_pool, req.max_frames
923             )
924     except Exception:
925         # A raising segmenter must not leave half-written rows: restore the pre-run
926         # state (same destroy-after-confirm contract as the Failure branch), then
let
927         # the job worker record the failure.
928         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)

```

```

929         raise
930     segmentation_ran = True
931
932     if isinstance(result, Failure):
933         logger.error(f"[API] Segmenter failed: {result.message}")
934         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
935         response = {
936             "video_id": video_id,
937             "status": "failed",
938             "message": f"Segmenter '{seg_name}' failed: {result.message}",
939             "results": [r.model_dump() for r in val_results],
940             "play_segments": [],
941         }
942         return response
943
944     segments = result.value if hasattr(result, "value") else result
945     notify("finalizing")
946     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
947
948     response = {
949         "video_id": video_id,
950         "status": "success",
951         "message": f"Successfully indexed {len(segments)} play segments using
952 '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
953         "results": [r.model_dump() for r in val_results],
954         "play_segments": segments,
955     }
956     return response
957
958     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
959     already set
960     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
961     other path stamp
962     # the actual path here so the job result PROVES which backend ran (in_process /
963     not_run) and
964     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
965     then visible.
966     if isinstance(response, dict):
967         response.setdefault(
968             "compute",
969             {
970                 "path": compute_actual or "not_run",
971                 "requested": getattr(req, "compute", None),
972             },
973         )
974     prov = response.get("compute", {}) if isinstance(response, dict) else {}
975     logger.info(
976         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
977         video_id,
978         getattr(req, "compute", None),
979         prov.get("path"),
980         prov.get("dispatched"),
981         (response or {}).get("status") if isinstance(response, dict) else None,
982     )
983     # Always emit the per-video observability report (next to the video, or to
984     # report.output_dir). Never raises. Surface the paths in the response. (The
985     compute-path
986     # proof lives on the job result + the [COMPUTE] log, not the report ?
987     obsreport's record_run
988     # doesn't surface arbitrary run signals.)
989     paths = _main.emit_indexer_report(
990         db=db,
991         video_id=video_id,
992         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to

```

```

the
986         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
987         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
988         # still preserved in DB provenance via db.add_video.
989         video_path=local_video,
990         config=_report_config_for_input(config, req.video_path),
991         val_results=val_results,
992         val_context=val_context,
993         segmenter_requested=req.segmenter,
994         segmenter_actual=segmenter_actual,
995         was_reingest=was_reingest,
996         segmentation_ran=segmentation_ran,
997     )
998     if paths and isinstance(response, dict):
999         response["reports"] = paths
1000     storage.cleanup_acquired_local_input(
1001         req.video_path, local_video, getattr(config, "storage", None)
1002     )
1003
1004
1005     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1006         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1007
1008         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
1009         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
1010         to a no-op rather than blocking a job it cannot measure."""
1011         try:
1012             sc = (
1013                 storage_config.model_dump()
1014                 if hasattr(storage_config, "model_dump")
1015                 else (storage_config or {})
1016             )
1017             backend = storage.get_storage(input_ref.backend, config=sc)
1018             return int(backend.stat(input_ref).size)
1019         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
1020             logger.debug(
1021                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
1022             )
1023             return None
1024
1025
1026     class _CompileError(Exception):
1027         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
1028         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
1029
1030         def __init__(self, status_code: int, detail: str):
1031             super().__init__(detail)
1032             self.status_code = status_code
1033             self.detail = detail
1034
1035
1036     def _resolve_compile(
1037         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
1038     ):
1039         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
1040         row could change between submit and run). Verifies the video exists, the requested
segments match

```

```

1041         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
1042         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
1043         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``. """
1044         video = db.get_video(video_id)
1045         if not video:
1046             raise _CompileError(404, "Indexed video record not found.")
1047         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
1048         try:
1049             out_name = safe_output_filename(output_filename)
1050         except ValueError as e:
1051             raise _CompileError(400, str(e)) from e
1052
1053         all_segments = db.query_play_segments(video_id, {})
1054         matched = [s for s in all_segments if s["id"] in segment_ids]
1055         if not matched:
1056             raise _CompileError(400, "No matching play segments found to stitch.")
1057
1058         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1059         kept = []
1060         for s in matched:
1061             meta = s.get("metadata") or {}
1062             sc = meta.get("shot_count") if isinstance(meta, dict) else None
1063             try:
1064                 if sc is not None and int(sc) < stitching.min_shot_count:
1065                     continue
1066             except (TypeError, ValueError):
1067                 pass # unparseable count -> treat as unknown, keep
1068             kept.append(s)
1069         if len(kept) < len(matched):
1070             logger.info(
1071                 "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1072                 stitching.min_shot_count,
1073                 len(matched) - len(kept),
1074                 len(matched),
1075             )
1076         if not kept:
1077             raise _CompileError(
1078                 400,
1079                 f"All {len(matched)} matching segments fall below "
1080                 f"stitching.min_shot_count={stitching.min_shot_count}.",
1081             )
1082         return video, kept, out_name
1083
1084
1085     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1086         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
1087         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1088         mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``. """
1089         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1090         if not root or "://" not in root:
1091             return ""
1092         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1093
1094
1095     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1096         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,

```

```

1097         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1098         isn't silently constructed as a fresh default.""
1099         sc = getattr(cfg, "storage", None)
1100         return sc.model_dump() if sc is not None else None
1101
1102
1103     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1104         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1105         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1106         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1107         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1108         reel_uri = reel_object_uri(cfg, video_id, name)
1109         if not reel_uri:
1110             return None
1111         settings = _storage_settings(cfg)
1112         if storage.exists(reel_uri, config=settings):
1113             return storage.signed_url(reel_uri, config=settings, method="GET")
1114         return None
1115
1116
1117     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1118         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1119         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1120         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1121         so the SPA always gets a clean verdict instead of a worker-crash error."""
1122         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1123
1124         notify = progress or (lambda stage: None)
1125         params = json.loads(job.get("params") or "{}")
1126         video_id = job.get("video_id") or ""
1127         segment_ids = params.get("segment_ids") or []
1128         locale = params.get("locale")
1129
1130         notify("resolving")
1131         try:
1132             video, kept, out_name = _resolve_compile(
1133                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1134             )
1135         except _CompileError as e:
1136             return {"status": "failed", "message": e.detail, "video_id": video_id}
1137
1138         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1139         output_dir = (
1140             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1141         )
1142         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1143         # unknown/absent locale keeps the configured default unchanged.
1144         localized_watermark = localize_watermark(stitching.watermark, locale)
1145         compiler = _main.VideoCompiler(
1146             output_dir,
1147             begin_padding=stitching.begin_padding,
1148             end_padding=stitching.end_padding,
1149             watermark=localized_watermark,
1150         )
1151         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1152

```

```

1153     notify("stitching")
1154     local_src = ""
1155     try:
1156         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1157         # path). Resolve to
1158         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1159         # scratch for a URI (the
1160         # same seam the process path uses); without it, compiling a bucket-ingested
1161         # video can't open gs://.
1162         local_src = storage.acquire_local_input(
1163             video["filepath"], getattr(cfg, "storage", None)
1164         )
1165         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
1166         # (for remote
1167         # dispatch) doesn't run multiple compiles at once on the box (see
1168         # _LOCAL_COMPUTE_LANE).
1169         with _LOCAL_COMPUTE_LANE:
1170             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1171         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
1172             FAILURE, not a crash
1173             logger.error(
1174                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1175             )
1176             return {
1177                 "status": "failed",
1178                 "message": f"Highlight stitching failed: {e}",
1179                 "video_id": video_id,
1180             }
1181         finally:
1182             if local_src:
1183                 storage.cleanup_acquired_local_input(
1184                     video["filepath"], local_src, getattr(cfg, "storage", None)
1185                 )
1186             )
1187             # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
1188             # URL ? Cloud Run
1189             # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
1190             # copy dies when the
1191             # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
1192             # (the endpoint
1193             # still falls back to the local file on the same instance).
1194             reel_uri = reel_object_uri(cfg, video_id, out_name)
1195             if reel_uri:
1196                 try:
1197                     notify("uploading reel")
1198                     # Thread the active storage settings so the configured backend (S3
1199                     # endpoint/region, GCS
1200                     # project/client) is used, not a fresh default.
1201                     storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1202                     except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
1203                         already succeeded
1204                         logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1205                     # `filepath` is server-local (not browser-reachable); also return the download URL
1206                     # so the SPA needn't
1207                     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1208                     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1209                     notify("finished")
1210                     return {
1211                         "status": "success",
1212                         "filepath": out_path,
1213                         "download_url": download_url,
1214                         "video_id": video_id,
1215                     }
1216                 }
1217             }
1218         }
1219     }

```

4. assistant (2026-07-08T05:45:15.719Z)

Now let me follow the imports across files.

5. user (2026-07-08T05:45:17.121Z)

```
1      """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
2      P3).
3      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6      re-claimed when due ? with no central scheduler (the DB is the clock).
7
8      The monkeypatch contract these functions rely on (and that tests depend on):
9      every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10     `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11     `backend.main` module object at call-time (`import backend.main as _main`), never as a
12     bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13     ...)` still intercepts the inter-function calls (e.g. a parked job arms the `patched`
14     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
15     lazily
16     inside the functions also avoids the circular import at module load (main.py imports
17     this
18     module; this module must not import main at import time).
19     """
20
21     import json
22     import logging
23     import os
24     import threading
25     import traceback
26     from concurrent.futures import ThreadPoolExecutor
27     from typing import Any, Dict, Optional
28
29     logger = logging.getLogger(__name__)
30
31     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
32     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
33     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
34     # already parallelize their AI handoff internally via their own pools.
35     # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
36     _cloud_dispatch_possible), a REMOTE
37     # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
38     Gemini), so the drain
39     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
40     parallelize local
41     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
42     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
43     invariant holds
44     # and compile/local-override jobs stay serial. The count is fixed at first
45     _get_executor() creation.
46
47     #
48     # ? O2 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
49     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
50     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
51     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
52     #   - detector timeout_sec kills hung WSL/GPU calls
53     #   - Cloud Run poll has a max_wait_sec bound
54     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
55     # A future slice should add a per-job wall-clock timeout (the executor itself
56     # cannot enforce timeouts on the Thread).
57     _job_executor: Optional[ThreadPoolExecutor] = None
58     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
59     configure_drain()
```



```

51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55
56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor

```

```

102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
108         `set_job_waiting`),
109         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
110         date
111         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
112         contract.
113
114         The wiring is additive: the production segmenter path never raises this today (zero
115         behaviour change), so a job runs exactly as before. The hook is what a later slice
116         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
117         """
118
119         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
120             super().__init__(f"job parked for {delay_sec:.0f}s")
121             self.delay_sec = max(0.0, float(delay_sec))
122             self.progress = progress
123
124     def _job_to_request(job: Dict[str, Any]):
125         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
126         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
127         the original submit. The row stores exactly the ProcessRequest fields."""
128         import backend.main as _main
129
130         raw_params = job.get("params")
131         params = (
132             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
133         )
134         return _main.ProcessRequest(
135             video_path=job["video_path"],
136             player_pool=job.get("player_pool"),
137             max_frames=job.get("max_frames"),
138             segmenter=job.get("segmenter"),
139             # v8 compute-picker: the worker runs _execute_processing ?
140             _maybe_dispatch_remote, which
141             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
142             only read off
143             # the job row for analytics). NULL ? server default = pre-picker behaviour.
144             compute=job.get("compute"),
145             force=bool(params.get("force")),
146         )
147
148     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
149         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
150         its
151         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
152
153         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
154         means
155         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
156         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
157         the
158         job self-scheduled and will be re-claimed when `next_poll_at` is due.
159         """
160         import backend.main as _main
161
162         job_id = job["id"]
163         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
164         (NULL/default ? the

```

```

158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:
162             if is_compile:
163                 result = _main._execute_compile(
164                     config,
165                     db,
166                     job,
167                     progress=lambda stage: db.set_job_progress(job_id, stage),
168                 )
169             else:
170                 req = _main._job_to_request(job)
171                 result = _main._execute_processing(
172                     config,
173                     db,
174                     req,
175                     progress=lambda stage: db.set_job_progress(job_id, stage),
176                 )
177             if result.get("video_id"):
178                 db.set_job_video_id(job_id, result["video_id"])
179             db.mark_job_done(job_id, result)
180             if not is_compile:
181                 _maybe_record_job_cost(
182                     job, result, config, db
183                 ) # M8 cost ledger (default-OFF; process jobs only)
184                 _maybe_run_flywheel(
185                     job, result, config
186                 ) # M11 data-flywheel (process jobs only)
187         except _main.JobWaiting as w:
188             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190             db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191         except Exception as e:
192             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193             db.mark_job_failed(job_id, f"type(e).__name__: {e}")
194
195
196     def _maybe_record_job_cost(
197         job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198     ) -> None:
199         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
202         persist it.
203
204         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
205         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
206         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
207         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
208         With the placeholder pricebook the cost is 0 regardless.
209
210         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
211         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
212
213         billing_cfg = getattr(config, "billing", None)
214         if not billing_cfg or not getattr(billing_cfg, "enabled", False):

```

```

215         return
216     try:
217         usage = (result or {}).get("usage") or {}
218         db.record_job_cost(
219             job["id"],
220             usage=usage,
221             pricebook_version=billing_cfg.pricebook_version,
222             gpu_type=usage.get("gpu_type"),
223             compute_backend=(result or {}).get("compute_backend"),
224             owner_id=job.get("owner_id"),
225             margin=billing_cfg.margin,
226         )
227         logger.info(
228             "Recorded cost for job %s (pricebook=%s).",
229             job["id"],
230             billing_cfg.pricebook_version,
231         )
232     except Exception as e: # never wedge a completed job on bookkeeping
233         logger.warning(
234             "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235         )
236
237
238     def _maybe_run_flywheel(
239         job: Dict[str, Any], result: Dict[str, Any], config: Any
240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output."""
249
250         flywheel_cfg = getattr(config, "flywheel", None)
251         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252             return
253         try:
254             from backend import flywheel
255
256             flywheel.run_for_job(job, result, config)
257         except Exception as e: # never wedge a completed job on the flywheel
258             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261     def _drain_due_jobs(config: Any, db: Any) -> int:
262         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267         a single-worker box keeps making progress with no central timer (the DB is the
clock).
268
269         Returns the number of jobs that reached a terminal/parked state this tick.
270         """
271         import backend.main as _main

```

```

271
272     ran = 0
273     while True:
274         job = db.claim_due_job()
275         if job is None:
276             break
277         ran += 1
278         _main._run_claimed_job(job, config, db)
279     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281     # on the next submit/drain. Best-effort ? never raises into the worker.
282     _main._schedule_next_drain_if_waiting(config, db)
283     return ran
284
285
286     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288         self-scheduled job resumes with no further client submit. The drain itself re-checks
289         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:
293             delay = db.seconds_until_next_due()
294         except Exception as e: # pragma: no cover - defensive; never wedge the worker
295             logger.error(f"Could not compute next-due delay: {e}")
296             return
297         if delay is None:
298             return # nothing waiting
299         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
307         import backend.main as _main
308
309         timer = threading.Timer(
310             delay_sec,
311             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312         )
313         timer.daemon = True
314         timer.start()
315
316
317     #: A parked job further out than this is re-checked by a capped wake rather than one
long
318     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319     _MAX_DRAIN_WAKE_SEC = 60.0
320
321
322     #
-----
323     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
324     # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart

```

```

325     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
326     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
327     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
328     # re-dispatching (no double GPU spend).
329
330     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331     _RESUME_POLL_EVERY_SEC = 15.0
332
333
334     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
337         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
338         thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
339         NEVER re-dispatched from here. Returns the number of resumes started.
340
341         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
342         exist ? the rows get today's honest terminal failure instead."""
343         rows = db.interrupted_remote_process_jobs()
344         if not rows:
345             return 0
346         if not _cloud_dispatch_possible(config):
347             for job in rows:
348                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349                 logger.warning(
350                     "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                     "dispatch to Cloud Run, so no remote worker can be in flight.",
352                     len(rows),
353                 )
354             return 0
355         # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
356         # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
357         # default target itself is cloud_run (on a default-local server with cloud-override
358         # capability, a NULL row ran in-process ? no remote worker exists to resume).
359         default_is_cloud = (
360             getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
361         )
362         started = 0
363         for job in rows:
364             # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
365             # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
366             requested = (str(job.get("compute") or "").strip().lower())
367             if requested != "cloud" and not default_is_cloud:
368                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
369                 continue
370             t = threading.Thread(
371                 target=_resume_one_remote_job,
372                 args=(config, db, dict(job)),
373                 daemon=True,
374                 name=f"resume-{str(job['id'][:8])}",
375             )
376             t.start()
377             started += 1
378         if started:

```

```

379         logger.warning(
380             "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
381             started,
382         )
383     return started
384
385
386 def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387     """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
388     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389     fail the job HONESTLY from what the bucket says. Never dispatches."""
390     from backend import orchestration, storage
391     from backend.infrastructure.execution_policy import (
392         ExecutionPolicy,
393         PolicyTimeout,
394         poll_until,
395     )
396
397     job_id = str(job["id"])
398     video_id = str(job["video_id"])
399     try:
400         raw_params = job.get("params")
401         params = (
402             raw_params
403             if isinstance(raw_params, dict)
404             else json.loads(raw_params or "{}")
405         )
406         if params.get("force"):
407             # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
408             # but a forced job must NEVER complete from the bucket cache ? force
deliberately
409             # bypasses it, and nothing proves the report belongs to the forced run
rather
410             # than an older completed one.
411             db.mark_job_failed(
412                 job_id,
413                 "control plane restarted during a forced reprocess; the cached result "
414                 "cannot be trusted for force=true ? re-submit with force",
415             )
416             return
417         if (str(job.get("compute") or "").strip().lower() == "local":
418             # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
419             # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
420             db.mark_job_failed(job_id, "interrupted by server restart")
421             return
422         sc = orchestration._storage_settings(config)
423         output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424         if not sc or not output_root:
425             db.mark_job_failed(
426                 job_id,
427                 "interrupted by server restart (no output bucket configured to recover
from)",
428             )
429             return
430         report_uri = f"{output_root}/outputs/{video_id}/report.json"
431         max_wait = float(
432             getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433             or 3900.0

```

```

434         )
435     policy = ExecutionPolicy(
436         poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437         op_timeout_sec=120.0,
438         max_wait_sec=max_wait,
439     )
440     db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441     try:
442         poll_until(
443             lambda: True if storage.exists(report_uri, config=sc) else None,
444             policy,
445             label=f"resume-{job_id[:8]}",
446             logger=logger,
447             # The #493 live heartbeat: job.progress moves once per still-waiting
448             on_tick=lambda elapsed: db.set_job_progress(
449                 job_id,
450                 "recovering: waiting for the in-flight cloud run ? "
451                 f"{int(elapsed) // 60} min",
452             ),
453         )
454     except PolicyTimeout:
455         db.mark_job_failed(
456             job_id,
457             "control plane restarted mid-run and the cloud worker's result never "
458             "appeared within the poll budget; re-submit the video (a finished result "
459             "re-hydrates instantly)",
460         )
461         return
462     payload = orchestration.cached_process_result(
463         db, config, video_id, str(job.get("video_path") or "")
464     )
465     if payload is not None:
466         db.mark_job_done(job_id, payload)
467         logger.warning(
468             "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469             job_id,
470             video_id,
471         )
472         return
473     # The report exists but did not realize into a usable result ? surface its
474     verdict.
475     status = "unreadable"
476     try:
477         report = orchestration._read_bucket_json(report_uri, sc) or {}
478         status = str(report.get("status") or "unreadable")
479     except Exception: # noqa: BLE001 ? the failure message is best-effort
480         pass
481     db.mark_job_failed(
482         job_id,
483         f"cloud worker finished without a usable result after a control-plane
484         restart "
485         f"(report status: {status}); re-submit the video",
486     )
487     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
488     not vanish
489         logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
490         try:
491             db.mark_job_failed(job_id, f"restart recovery failed: {e}")
492         except Exception: # noqa: BLE001
493             pass

```



```

1      """Canonical file-identity hashing for the indexer.
2
3      `video_id` is the MD5 of the *whole* video file. Historically each entriypoint
4      (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5      `scripts/run_process_and_stitch.py`)
6      defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7      The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8      any future change to one copy would silently diverge the DB key.
9
10     This module is the single source of truth. Import `compute_video_id` everywhere a
11     video needs to be keyed.
12     """
13
14     from __future__ import annotations
15
16     import hashlib
17
18     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
19     # was centralized. The chunk size does not affect the resulting digest ? it only
20     # bounds memory use while streaming large files.
21     _CHUNK_SIZE = 65536
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
31             for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                 hasher.update(chunk)
33         return hasher.hexdigest()
34

```

7. user (2026-07-08T05:45:25.955Z)

```

53: def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
69: def input_is_local(raw: str, storage_config: Any = None) -> bool:
77: def acquire_local_input(
101: def cleanup_acquired_local_input(
170: def fetch(
185: def put(

```

8. user (2026-07-08T05:45:26.974Z)

```

26: class ValidationResult(BaseModel):
33: class VideoValidator(ABC):
36:     def name(self) -> str:
42:     def description(self) -> str:
47:     def validate(
61: class FrameSamplingValidator(VideoValidator):
73:     a subclass is invoked standalone (direct ``.validate`` call, no shared
80:     def sample_indices(self, total_frames: int, fps: float) -> List[int]:
90:     def _init_state(self, config: Any) -> Any:
95:     def _consume(self, state: Any, frame_idx: int, frame: Any) -> None:
100:     def _finalize(
106:     def validate(
121: class ValidationRegistry:
122:     def __init__(self):
125:     def register(self, validator: VideoValidator):
129:     def run_all_with_context(self, video_path: str, config: Any):
156:     def _prepare_frame_samples(
183:     def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:

```

```
188:     def config_fingerprint(self, config: Any) -> str:
```

9. user (2026-07-08T05:45:34.931Z)

```
40     "resolve_input_ref",
41 ]
42
43
44     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
45         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
46         if storage_config is None:
47             return {}
48         if hasattr(storage_config, "model_dump"):
49             return storage_config.model_dump() # type: ignore[no-any-return]
50         return dict(storage_config)
51
52
53     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
54         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
55         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
56         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
57
58         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
59         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
60         sc = _storage_dict(storage_config)
61         uri = resolve_input_uri(
62             raw,
63             input_backend=sc.get("input_backend", "local"),
64             input_root=sc.get("input_root", ""),
65         )
66         return parse_uri(uri)
67
68
69     def input_is_local(raw: str, storage_config: Any = None) -> bool:
70         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
71         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
72         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
73         ``resolve_input_ref``)."""
74         return resolve_input_ref(raw, storage_config).backend == "local"
75
76
77     def acquire_local_input(
78         raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
79     ) -> str:
80         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
81
82         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
83         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``drive://``, or a
bare name under a
84         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
85         path returned. The caller owns successful remote scratch copies; use
86         ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
87         sc = _storage_dict(storage_config)
88         ref = resolve_input_ref(raw, sc)
```

```

89         if ref.backend == "local":
90             return (
91                 ref.key
92             ) # identity ? no temp, no copy, byte-identical to passing the path directly
93         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
94         try:
95             return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
96         except Exception:
97             shutil.rmtree(scratch, ignore_errors=True)
98             raise
99
100
101     def cleanup_acquired_local_input(
102         raw: str, local_path: str, storage_config: Any = None
103     ) -> None:
104         """Delete a remote scratch dir produced by ``acquire_local_input``.
105
106         Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
107         into a fresh ``rally-input-`` directory; deleting that directory is safe once the
caller has
108         finished hashing/validating/stitching/segmenting the returned file.
109         """
110         try:
111             if input_is_local(raw, storage_config):
112                 return
113         except ValueError:
114             return
115         if not local_path:
116             return
117         scratch = os.path.dirname(os.path.abspath(local_path))
118         if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
119             shutil.rmtree(scratch, ignore_errors=True)
120
121
122     def _backend_class(name: str):
123         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
124         if name == "local":
125             from backend.storage.local import LocalStorage
126
127             return LocalStorage
128         if name == "s3":
129             from backend.storage.s3 import S3Storage
130
131             return S3Storage
132         if name == "gcs":
133             from backend.storage.gcs import GCSStorage
134
135             return GCSStorage
136         if name == "gdrive":
137             from backend.storage.gdrive import GDriveStorage
138
139             return GDriveStorage
140         if name == "gdrive-api":
141             from backend.storage.gdrive_api import GDriveApiStorage
142
143             return GDriveApiStorage
144         raise ValueError(
145             f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
146         )
147
148
149     def get_storage(
150         backend: Optional[str] = None,
151         *,

```

```

152         config: Optional[Dict[str, Any]] = None,
153         **overrides,
154     ) -> StorageBackend:
155         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
156         the implementation; its per-backend settings come from ``config[backend]`` merged
with
157         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
158         cfg = config or {}
159         name = backend or cfg.get("backend", "local")
160         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
161         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
162         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
163         settings: Dict[str, Any] = dict(
164             cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
165         )
166         settings.update(overrides)
167         return _backend_class(name)(**settings)
168
169
170     def fetch(
171         uri: str,
172         dst: "str | os.PathLike[str] | None" = None,
173         *,
174         config: Optional[Dict[str, Any]] = None,
175         overwrite: bool = False,
176     ) -> str:
177         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
178         (no copy); for s3/gcs it downloads to ``dst``."""
179         ref = parse_uri(uri)
180         return get_storage(ref.backend, config=config).fetch_to_local(
181             ref, dst, overwrite=overwrite
182         )
183
184
185     def put(
186         local_path: "str | os.PathLike[str]",
187         uri: str,
188         *,
189         config: Optional[Dict[str, Any]] = None,
190         content_type: Optional[str] = None,
191     ) -> str:
192         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
193         ref = parse_uri(uri)
194         get_storage(ref.backend, config=config).put(
195             local_path, ref, content_type=content_type
196         )
197         return uri
198
199

```

10. user (2026-07-08T05:45:35.700Z)

```

1     import hashlib
2     import json
3     import logging
4     from abc import ABC, abstractmethod
5     from typing import Any, List
6
7     from pydantic import BaseModel
8

```

```

9      from backend.pipeline.validators.frame_sampler import (
10          FRAME_META_KEY,
11          FRAME_STATES_KEY,
12          SampleMeta,
13          run_frame_sampling,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bumped when validator LOGIC (not config) changes the verdict for identical
19      bytes+config ? e.g. a
20      # validator is rewritten, or the sampling arithmetic changes. It rides inside
21      config_fingerprint, so
22      # a bump invalidates every cached validation verdict (see backend/orchestration.py's
23      validation cache
24      # + backend/infrastructure/database.py v10). The validator SET and each validator's
25      config-derived
26      # thresholds are already folded into the fingerprint; this covers changes the config
27      can't express.
28      _VALIDATION_FINGERPRINT_SCHEMA = 1
29
30      class ValidationResult(BaseModel):
31          validator_name: str
32          passed: bool
33          message: str
34          details: dict[str, Any] = {}
35
36      class VideoValidator(ABC):
37          @property
38          @abstractmethod
39          def name(self) -> str:
40              """Unique identifier for this validator."""
41              pass
42
43          @property
44          @abstractmethod
45          def description(self) -> str:
46              """Human-readable explanation of what this validator checks."""
47              pass
48
49          @abstractmethod
50          def validate(
51              self, video_path: str, config: Any, context: dict[str, Any]
52          ) -> ValidationResult:
53              """
54              Executes validation.
55              - video_path: Local path to the MP4 file.
56              - config: AppConfig (typed) ? callers pass the typed model; attribute access
57              preferred over .get() dict chains.
58              - context: Dynamic context dict shared across validators (stores shared values
59              like ffp probe metadata).
60              """
61              pass
62
63      class FrameSamplingValidator(VideoValidator):
64          """A validator whose work is sampling decoded frames.
65
66          The three frame-decoding validators (scene_cut, blur, relevance) used to each
67          open their own ``cv2.VideoCapture`` and seek to their sample frames ? ~125
68          random ``POS_FRAMES`` seeks across three decode opens, which cost minutes on
69          long-GOP 4K clips (see ``frame_sampler``). Subclasses instead declare *which*
70          frame indices they want (:meth:`sample_indices`) and reduce those frames

```

```

69     online (:meth:`_init_state` / :meth:`_consume`), so the registry can decode
70     the whole set **once** in a single shared forward pass.
71
72     :meth:`validate` uses the shared pass' result when the registry ran it; when
73     a subclass is invoked standalone (direct ``.validate`` call, no shared
74     context) it decodes its own indices in one pass via the same engine. Either
75     way the frames ? and therefore the pass/fail decision ? are identical to the
76     old per-validator seeks.
77     """
78
79     @abstractmethod
80     def sample_indices(self, total_frames: int, fps: float) -> List[int]:
81         """Absolute frame indices this validator samples, in ascending order.
82
83         Pure and deterministic in ``(total_frames, fps)`` ? the shared pass unions
84         these across validators to drive one decode, and the regression suite pins
85         them to the legacy per-validator seek arithmetic.
86         """
87         ...
88
89     @abstractmethod
90     def _init_state(self, config: Any) -> Any:
91         """Fresh accumulator for one run (may pre-resolve config-derived params)."""
92         ...
93
94     @abstractmethod
95     def _consume(self, state: Any, frame_idx: int, frame: Any) -> None:
96         """Fold one sampled frame (BGR ndarray) into ``state``. Called in index
97         order."""
98         ...
99
100    @abstractmethod
101    def _finalize(
102        self, state: Any, meta: SampleMeta, config: Any, context: dict[str, Any]
103    ) -> ValidationResult:
104        """Render the ValidationResult from the accumulated ``state`` + decode meta."""
105        ...
106
107    def validate(
108        self, video_path: str, config: Any, context: dict[str, Any]
109    ) -> ValidationResult:
110        states = context.get(FRAME_STATES_KEY)
111        if states is not None and self.name in states:
112            # Shared single pass already decoded and reduced our frames.
113            meta = context[FRAME_META_KEY]
114            state = states[self.name]
115        else:
116            # Standalone: decode just this validator's indices in one pass.
117            meta, own_states = run_frame_sampling(video_path, [self], config)
118            state = own_states[self.name]
119        return self._finalize(state, meta, config, context)
120
121    class ValidationRegistry:
122        def __init__(self):
123            self._validators: List[VideoValidator] = []
124
125        def register(self, validator: VideoValidator):
126            """Registers a new validator."""
127            self._validators.append(validator)
128
129        def run_all_with_context(self, video_path: str, config: Any):
130            """Runs all validators and returns ``(results, context)``.
131
132            The per-run ``context`` accumulates shared values (notably

```

```

133         ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
134         discards it; callers that want media metadata for an observability report
135         use this variant instead of re-probing.
136         """
137         results = []
138         context: dict[
139             str, Any
140         ] = {} # Shared context (e.g. for caching ffprobe outputs)
141         self._prepare_frame_samples(video_path, config, context)
142         for val in self._validators:
143             try:
144                 res = val.validate(video_path, config, context)
145                 results.append(res)
146             except Exception as e:
147                 results.append(
148                     ValidationResult(
149                         validator_name=val.name,
150                         passed=False,
151                         message=f"Validator threw an unexpected error: {str(e)}",
152                     )
153                 )
154         return results, context
155
156     def _prepare_frame_samples(
157         self, video_path: str, config: Any, context: dict[str, Any]
158     ) -> None:
159         """Decode every frame-sampling validator's frames in ONE shared pass.
160
161         Populates ``context`` with the shared decode meta + per-validator
162         accumulator state; each :class:`FrameSamplingValidator` then finalizes
163         from that instead of opening its own capture. Best-effort: any failure
164         here is swallowed so validators transparently fall back to a standalone
165         pass (the run must never crash on a decode hiccup)."""
166         samplers = [
167             v for v in self._validators if isinstance(v, FrameSamplingValidator)
168         ]
169         if not samplers:
170             return
171         try:
172             meta, states = run_frame_sampling(video_path, samplers, config)
173         except Exception:
174             logger.exception(
175                 "Shared frame-sampling pass failed for %s; validators will fall "
176                 "back to standalone decode.",
177                 video_path,
178             )
179         return
180         context[FRAME_META_KEY] = meta
181         context[FRAME_STATES_KEY] = states
182
183     def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:
184         """Runs all registered validators in sequence, accumulating results."""
185         results, _ = self.run_all_with_context(video_path, config)
186         return results
187
188     def config_fingerprint(self, config: Any) -> str:
189         """Stable hash of EVERYTHING that determines a validation verdict for fixed
190         video bytes.
191
192         Powers the video_id-keyed validation-result cache (backend/orchestration.py): a
193         reupload of
194         identical content reuses its stored verdict ONLY while this fingerprint still
195         matches, so any
196         change that could flip a decision correctly forces a re-validation (never serve
197         a stale

```

```

194         PASS/FAIL). What goes in ? and why each matters:
195
196         * the whole ``validation`` config block ? every threshold the validators read
197         (min_blur_threshold / min_fps / min_resolution / max_scene_cuts_per_minute /
pts gap /
198         relevance court-color bounds). Hashing the full model (not a hand-picked
subset) means a
199         NEW validation knob is covered automatically.
200         * ``sport`` ? the relevance validator keys its court-color profile off
``config.sport`` (a
201         top-level field, NOT under ``validation``), so switching sports changes the
verdict.
202         * the sorted validator NAMES ? adding/removing a validator changes which checks
run.
203         * a schema version (:data:_VALIDATION_FINGERPRINT_SCHEMA`) ? a manual escape
hatch for
204         validator LOGIC changes the config can't express.
205
206         Accepts a typed ``AppConfig`` (attribute access) or a plain dict;
``default=str`` keeps the
207         dump total on any exotic value. Deterministic: ``sort_keys`` fixes ordering."""
208         validation: Any = getattr(config, "validation", None)
209         if validation is None and isinstance(config, dict):
210             validation = config.get("validation")
211         validation_repr = (
212             validation.model_dump(mode="json")
213             if hasattr(validation, "model_dump")
214             else validation
215         )
216         sport: Any = getattr(config, "sport", None)
217         if sport is None and isinstance(config, dict):
218             sport = config.get("sport")
219         payload = {
220             "schema": _VALIDATION_FINGERPRINT_SCHEMA,
221             "validators": sorted(v.name for v in self._validators),
222             "sport": sport,
223             "validation": validation_repr,
224         }
225         blob = json.dumps(
226             payload, sort_keys=True, default=str, separators=(",", ":")
227         )
228         return hashlib.sha256(blob.encode("utf-8")).hexdigest()
229
230
231     # Shared global registry
232     registry = ValidationRegistry()
233

```

11. user (2026-07-08T05:45:44.061Z)

```

backend\pipeline\validators\__init__.py:10:registry.register(FormatValidator())
backend\pipeline\validators\__init__.py:11:registry.register(ResolutionFPSValidator())
backend\pipeline\validators\__init__.py:12:registry.register(PTSContinuityValidator())
backend\pipeline\validators\__init__.py:13:registry.register(SceneCutDensityValidator())
backend\pipeline\validators\__init__.py:14:registry.register(BlurValidator())
backend\pipeline\validators\__init__.py:15:registry.register(RelevanceValidator())

```

12. user (2026-07-08T05:45:44.639Z)

```

1     """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m

```



```

backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11        job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12        worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13        the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15        Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16        worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17        (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23
24    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
25    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26    owner-verified on the M7 real run.
27    ""
28
29    from __future__ import annotations
30
31    import json
32    import logging
33    import os
34    import tempfile
35    import uuid
36    from abc import ABC, abstractmethod
37    from typing import Any, Callable, List, Optional
38
39    from backend.compute.base import (
40        ComputeBackend,
41        ComputeJobSpec,
42        ComputeResult,
43        RemoteExecutionFailed,
44    )
45    from backend.infrastructure.execution_policy import (
46        DEFAULT_POLICY,
47        ExecutionPolicy,
48        PolicyTimeout,
49        poll_until,
50    )
51    from backend.storage import fetch, get_storage, parse_uri
52
53    LOG = logging.getLogger("compute.cloud_run")

```

```

54
55     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
56     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
57     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
58         "deployment.compute_target=local_gpu",
59         "indexing.detector_impl=native",
60     )
61
62
63     class CloudRunJobClient(ABC):
64         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
65
66         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
67         overrides the job's container ``args`` for this execution and starts it."""
68
69         @abstractmethod
70         def run_job(
71             self,
72             job_name: str,
73             argv: List[str],
74             *,
75             region: str,
76             project: Optional[str] = None,
77         ) -> str:
78             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
79             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
80
81         @abstractmethod
82         def cancel_execution(self, execution: str) -> None:
83             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
84             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
85             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
86             NEVER masked by a cancel error)."""
87
88         def execution_terminal_state(self, execution: str) -> Optional[str]:
89             """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
90             ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
91             can't be determined).
92
93             ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
94             terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
95             of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
96
97             NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
98             chooses not to ? poll the Executions API degrades transparently to today's
marker-only
99             polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
100            as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""

```

```

101         return None
102
103
104     class GoogleCloudRunJobClient(CloudRunJobClient):
105         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
106         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
107
108     def run_job(
109         self,
110         job_name: str,
111         argv: List[str],
112         *,
113         region: str,
114         project: Optional[str] = None,
115     ) -> str:
116         # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
117         # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
118         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
119         if not project:
120             raise RuntimeError(
121                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
122                 "to resolve the job path; set it on the control plane (see
get_compute_backend)."
123             )
124         try:
125             from google.cloud import run_v2 # type: ignore[attr-defined]
126         except Exception as exc: # pragma: no cover - real SDK not present in CI
127             raise RuntimeError(
128                 "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
129                 "control plane (it is intentionally NOT a CI/test dependency)."
130             ) from exc
131         client = (
132             run_v2.JobsClient()
133         ) # pragma: no cover - exercised only on the real M7 run
134         name = client.job_path(project, region, job_name)
135         overrides = run_v2.RunJobRequest.Overrides(
136             container_overrides=[
137                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
138             ]
139         )
140         op = client.run_job(
141             request=run_v2.RunJobRequest(name=name, overrides=overrides)
142         )
143         return (
144             getattr(op, "metadata", None)
145             and getattr(op.metadata, "name", "")
146             or "execution"
147         )
148
149     def cancel_execution(self, execution: str) -> None:
150         # run_job falls back to the literal string "execution" when the operation
metadata carries
151         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
152         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
153         # google-cloud-run is absent).
154         if "/executions/" not in (execution or ""):

```

```

155         raise RuntimeError(
156             f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
157             "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
158             "one from the operation metadata, so there is nothing to cancel by
name)."
159         )
160     try:
161         from google.cloud import run_v2 # type: ignore[attr-defined]
162     except Exception as exc: # pragma: no cover - real SDK not present in CI
163         raise RuntimeError(
164             "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
165             "control plane (it is intentionally NOT a CI/test dependency)."
166         ) from exc
167     client = (
168         run_v2.ExecutionsClient()
169     ) # pragma: no cover - exercised only on a real run
170     client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
171
172     def execution_terminal_state(self, execution: str) -> Optional[str]:
173         # Advisory status probe (see the ABC docstring): fetch the execution and
classify it via the
174         # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
absent SDK returns
175         # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
raises on bad
176         # input, because the probe is best-effort and must not fail a healthy run.
177         if "/executions/" not in (execution or ""):
178             return None
179         try:
180             from google.cloud import run_v2 # type: ignore[attr-defined]
181         except Exception: # pragma: no cover - real SDK not present in CI
182             return None
183         client = (
184             run_v2.ExecutionsClient()
185         ) # pragma: no cover - exercised only on a real run
186         exe = client.get_execution( # pragma: no cover - real run only
187             request=run_v2.GetExecutionRequest(name=execution)
188         )
189         return _classify_execution_state(exe) # pragma: no cover - real run only
190
191
192     def _classify_execution_state(exe: Any) -> Optional[str]:
193         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
``"Cancelled"``
194         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
SDK/network) so it is
195         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
? the actual
196         ``get_execution`` fetch is the only part that can't run in CI.
197
198         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
field is
199         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
to a proto
200         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
Terminality is
201         therefore ``completion_time`` is not None"; the per-task counts (plain ints) then
classify it,
202         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
0``)."""
203         if getattr(exe, "completion_time", None) is None:
204             return None # no terminal completion timestamp yet ? still pending/running

```

```

205     if getattr(exe, "cancelled_count", 0):
206         return "Cancelled"
207     if getattr(exe, "failed_count", 0):
208         return "Failed"
209     if getattr(exe, "succeeded_count", 0):
210         return "Succeeded"
211     return "Failed" # terminal but no positive success ? treat as failed (conservative)
212
213
214 class CloudRunCompute(ComputeBackend):
215     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
216
217     def __init__(
218         self,
219         *,
220         run_client: CloudRunJobClient,
221         job_name: str,
222         region: str,
223         project: Optional[str] = None,
224         storage_config: Optional[dict] = None,
225         policy: ExecutionPolicy = DEFAULT_POLICY,
226         token: Optional[str] = None,
227         container_overrides: Optional[tuple[str, ...]] = None,
228     ) -> None:
229         self._client = run_client
230         self._job_name = job_name
231         self._region = region
232         self._project = project
233         self._storage_config = storage_config or {}
234         self._policy = policy
235         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
236         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
237         self._token = token
238         self._container_overrides = (
239             _DEFAULT_CONTAINER_OVERRIDES
240             if container_overrides is None
241             else tuple(container_overrides)
242         )
243
244     def run_infer(
245         self,
246         spec: ComputeJobSpec,
247         *,
248         on_wait: "Callable[[float], None] | None" = None,
249     ) -> ComputeResult:
250         out_root = spec.out_uri.rstrip("/")
251         token = self._token or uuid.uuid4().hex
252         done_uri = f"{out_root}/_dispatch/{token}.done"
253         argv: List[str] = [
254             "infer",
255             "--video-uri",
256             spec.video_uri,
257             "--out-uri",
258             spec.out_uri,
259             "--done-uri",
260             done_uri,
261         ]
262         if spec.segmenter:
263             argv += ["--segmenter", spec.segmenter]
264         if spec.skip_validation:
265             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266             # re-decode and re-validate (redundant, and its config may resolve different

```

```

thresholds).
267         argv.append("--skip-validation")
268     for kv in (*spec.config_overrides, *self._container_overrides):
269         argv += ["--set", kv]
270
271     execution = self._client.run_job(
272         self._job_name, argv, region=self._region, project=self._project
273     )
274     LOG.info(
275         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
276         self._job_name,
277         execution,
278         done_uri,
279     )
280
281     try:
282         marker = poll_until(
283             lambda: self._poll_check(done_uri, str(execution or "")),
284             self._policy,
285             label="cloud-run-infer",
286             logger=LOG,
287             # Live heartbeat (#482): each still-waiting tick reaches the caller so
288             # job.progress moves during the whole GPU run instead of freezing.
289             on_tick=on_wait,
290         )
291     except PolicyTimeout as timeout_exc:
292         # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
293         # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
294         # clearly-labelled failure instead of the ~65-min opaque timeout this branch
handles.
295         marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
296         video_id = str(marker.get("video_id", ""))
297         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
298         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
299         rc = int(marker.get("returncode", 1))
300         if not video_id:
301             LOG.warning(
302                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
303                 "video_id) ? treating as failure",
304                 self._job_name,
305             )
306         rc = rc or 1
307         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
308         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
309         LOG.info(
310             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
311         )
312         return ComputeResult(
313             returncode=rc,
314             outputs_uri=outputs_uri,
315             video_id=video_id,
316             report=report,
317             execution=str(execution or ""),
318         )
319
320     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
321         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
322
323         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always

```

```

324 wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
325 FAILURE* marker resolves through the normal path with its real returncode. Only
when the
326 marker is absent do we consult the execution status: a worker that TERMINATED
without ever
327 writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
328 otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
329 opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
330 failure.
331
332 Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
333 ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
334 will ever appear)."""
335 marker = self._read_marker(done_uri)
336 if marker is not None:
337     return marker # success signal ? authoritative even if status later flips
338 state = self._terminal_execution_state(execution)
339 if state is None:
340     return None # still pending/running, or status unknown ? keep bucket-
polling
341 # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
342 # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
343 # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
344 # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
345 marker = self._read_marker(done_uri)
346 if marker is not None:
347     return marker
348 raise RemoteExecutionFailed(
349     f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
350     f"terminal state {state!r} with NO completion marker ? the worker exited
before "
351     "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
352     "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
353     "inspect the Cloud Run Job execution log to diagnose: "
354     f"`gcloud run jobs executions describe {execution or '<execution>'} "
355     f"--region {self._region}`."
356 )
357
358 def _terminal_execution_state(self, execution: str) -> Optional[str]:
359     """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
360     flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
361     so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
362     ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
363     if not execution:
364         return None
365     try:
366         return self._client.execution_terminal_state(execution)
367     except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
368         LOG.debug(

```

```

369         "cloud-run: execution status probe failed for %s ? treating as unknown",
370         execution,
371         exc_info=True,
372     )
373     return None
374
375     def _handle_poll_timeout(
376         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
377     ) -> dict:
378         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
379
380         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
381         job's own ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
382         the deadline: ONE final marker check rescues that completed GPU run instead of wasting it. If
383         the marker is genuinely absent, the execution is still running ? best-effort cancel it
384         (otherwise the L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
385         carrying the execution name so the operator can verify the state in the GCP console. A cancel
386         failure NEVER masks the original timeout."""
387         try:
388             late = self._read_marker(done_uri)
389         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
390             ? failure
391             LOG.warning(
392                 "cloud-run: final post-timeout marker check failed ? treating as
393                 absent",
394                 exc_info=True,
395             )
396             late = None
397             if late is not None:
398                 LOG.warning(
399                     "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
400                     check "
401                     "? rescuing the completed result",
402                     self._job_name,
403                     execution,
404                 )
405                 return late
406             try:
407                 self._client.cancel_execution(execution)
408             except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
409                 timeout
410                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
411                 LOG.warning(
412                     "cloud-run: best-effort cancel of execution=%s failed: %s",
413                     execution,
414                     cancel_exc,
415                     exc_info=True,
416                 )
417             else:
418                 cancel_note = "cancel requested"
419                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
420                 raise PolicyTimeout(
421                     f"{timeout_exc} [cloud-run job={self._job_name} "
422                     f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
423                     "
424                     "state in the GCP console]"
425                 ) from timeout_exc
426
427     # --- storage-backed completion signal (bucket-poll) ---

```



```

423     def _read_marker(self, done_uri: str) -> Optional[dict]:
424         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
425         ref = parse_uri(done_uri)
426         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
427             return None
428         return (
429             self._read_json(done_uri) or {}
430         ) # empty dict still terminates the poll (present == done)
431
432     def _read_json(self, uri: str) -> dict:
433         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
434         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
435         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
436         try:
437             ref = parse_uri(uri)
438             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
439                 dst = os.path.join(td, ref.name or "obj.json")
440                 local = fetch(uri, dst, config=self._storage_config)
441                 with open(local, "r", encoding="utf-8") as f:
442                     data: Any = json.load(f)
443                 return data if isinstance(data, dict) else {}
444         except Exception:
445             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
446             return {}
447

```

13. user (2026-07-08T05:45:50.904Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6     (cloud-serving) ? without the core ever branching on the profile (D1).
7
8     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9     non-local backend. Every other target (``"local_gpu" / "cpu_mock"``) keeps the
worker's
10    existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11    """
12
13    from __future__ import annotations
14
15    from abc import ABC, abstractmethod
16    from dataclasses import dataclass, field
17    from typing import Callable, Optional
18
19
20    class RemoteExecutionFailed(RuntimeError):
21        """A dispatched remote job's execution reached a TERMINAL state without ever writing
its
22        completion marker ? the worker crashed / OOM'd / aborted (e.g. an old image
argparse-rejecting
23        an unknown flag) before the marker push, or the marker upload itself failed on an
otherwise
24        Succeeded run.
25
26        Distinct from ``PolicyTimeout`` (poll budget exhausted while the execution may STILL
be

```

```

27         running): this says the execution is DEFINITELY over and no marker will ever
appear, so the
28         dispatcher must stop bucket-polling immediately instead of burning the full
29         ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message
carries the
30         job + execution name + terminal state + the ``gcloud`` command to read the execution
log, so
31         the whole "worker crashed before the done-marker" class fails fast AND
diagnostically."""
32
33
34     @dataclass(frozen=True)
35     class ComputeJobSpec:
36         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
37         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
38         deterministic regardless of where the job ran."""
39
40         video_uri: str # input (gs://? for cloud; a path/local:// for local)
41         out_uri: str # output ROOT (gs://? bucket for cloud)
42         segmenter: Optional[str] = None
43         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
44         config_overrides: tuple[str, ...] = ()
45         # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving
control-plane) is
46         # the authoritative validation gate: it runs the SAME validators with ITS resolved
config and only
47         # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48         # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49         # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50         # validating exactly as before.
51         skip_validation: bool = False
52
53
54     @dataclass(frozen=True)
55     class ComputeResult:
56         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
57         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
58         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
59         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60
61         returncode: int
62         outputs_uri: str = ""
63         video_id: str = ""
64         report: dict = field(default_factory=dict)
65         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
66         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
67         # executions describe <execution>). Empty for the in-process path / backends without
one.
68         execution: str = ""
69
70
71     class ComputeBackend(ABC):
72         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns

```

```

only once
73     its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
74     execution identically."""
75
76     @abstractmethod
77     def run_infer(
78         self,
79         spec: ComputeJobSpec,
80         *,
81         on_wait: "Callable[[float], None] | None" = None,
82     ) -> ComputeResult:
83         """Run ``spec`` to completion and return its ``ComputeResult``.
84
85         ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
86         still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
87         long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
88

```

14. user (2026-07-08T05:45:52.541Z)

```

42: def get_compute_backend(
43-     config: Any,
44-     *,
45-     run_client: Optional[Any] = None,
46-     policy: ExecutionPolicy = DEFAULT_POLICY,
47-     token: Optional[str] = None,
48-     target_override: Optional[str] = None,
49-) -> Optional[ComputeBackend]:
50-     """Return the compute backend for ``config.deployment.compute_target``, or ``None`` to
run
51-     in-process (the default-OFF path).
52-
53-     Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
54-     Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
55-     / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is injectable
(tests
56-     pass a fake; production defaults to the real ``google-cloud-run`` client, owner-verified
on M7).
57-
58-     ``target_override`` lets a single request force a target (the SPA compute-picker maps a
59-     per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating the
60-     server's configured default. ``None`` falls back to ``config.deployment.compute_target``.
61-     """
62-     target = (
63-         target_override
64-         if target_override is not None
65-         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
66-     )
67-     if target != "cloud_run":
68-         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
69-
70-     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
71-     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
72-
73-     # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
74-     # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout so a
75-     # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the old
76-     # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps an
77-     # explicitly-passed policy (tests, callers with bespoke budgets) untouched.

```

```

78-     if policy is DEFAULT_POLICY:
79-         max_wait = getattr(
80-             getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
81-         )
82-         if max_wait:
83-             policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
84-
85-     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
86-     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
87-     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
88-     client = run_client if run_client is not None else GoogleCloudRunJobClient()
89-     return CloudRunCompute(
90-         run_client=client,
91-         job_name=job_name,
92-         region=region,
93-         project=project,
94-         storage_config=_storage_dict(config),
95-         policy=policy,
96-         token=token,
97-     )

```

15. assistant (2026-07-08T05:45:59.109Z)

Now the submit path in backend/main.py ? the /api/process endpoint and the upload endpoint.

16. user (2026-07-08T05:46:00.979Z)

```

109: def create_app(config: AppConfig, db: Database) -> FastAPI:
180: @app.get("/", response_class=HTMLResponse)
181: def serve_index():
189: def _bundled_ui_version() -> Optional[Dict[str, Any]]:
252: def load_config(config_path: str = "config.json") -> Dict[str, Any]:
314: @app.get("/api/config")
315: def get_config(request: Request):
326: @app.get("/api/health")
327: def health(request: Request):
346: def _engine_version() -> str:
354: @app.get("/api/version")
355: def api_version():
364: @app.get("/api/capabilities")
365: def capabilities(request: Request):
414: def _friendly_video_name(filepath: str) -> str:
423: def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
436: def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
457: @app.get("/api/videos")
458: def list_videos(request: Request, limit: Optional[int] = None):
470: def _reels_dir(cfg) -> str:
477: @app.get("/api/reels")
478: def list_reels(request: Request):
510: @app.get("/api/reels/{filename}")
511: def download_reel(filename: str, request: Request):
541: def _proxies_dir(cfg) -> str:
545: def _source_cache_dir(cfg) -> str:
549: def _source_cache_lock(video_id: str) -> threading.Lock:
554: def _source_cache_path(video_id: str, input_ref, cfg) -> str:
563: def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
591: def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
614: @app.get("/api/source/{video_id}")
615: def get_source(video_id: str, request: Request):
674: @app.post("/api/annotations")
675: def post_annotations(req: AnnotationsRequest, request: Request):
712: def _storage_settings(cfg) -> Dict[str, Any]:
720: def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
725: def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
742: @app.post("/api/assets", status_code=201)

```

```

743: def create_asset(req: AssetRequest, request: Request):
904: @app.post("/api/process", status_code=202)
905: def process_video(req: ProcessRequest, request: Request):
977: @app.get("/api/jobs/{job_id}/cost")
978: def get_job_cost(job_id: str, request: Request):
992: @app.get("/api/videos/{video_id}/cost")
993: def get_video_cost(video_id: str, request: Request):
1003: @app.get("/api/jobs/{job_id}")
1004: def get_job(job_id: str, request: Request):
1041: @app.get("/api/highlights/{video_id}/{filename}")
1042: def download_highlight(video_id: str, filename: str, request: Request):
1082: @app.post("/api/query")
1083: def query_play_segments(req: QueryRequest, request: Request):
1098: @app.post("/api/compile", status_code=202)
1099: def compile_highlights(req: CompileRequest, request: Request):
1149: def run_cli_diagnose_clip(video_path: str, timestamp: float):
1232: def _enforce_startup_or_exit(config) -> None:
1247: def _maybe_sweep_gemini_orphans(config) -> None:
1271: def main():

```

17. user (2026-07-08T05:46:07.386Z)

```

742     @app.post("/api/assets", status_code=201)
743     def create_asset(req: AssetRequest, request: Request):
744         """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).
745
746         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
747         hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
748         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
749         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
750         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
751         stays as the working copy; a ``local`` medium keeps the upload in place (it's
already on this box);
752         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
753         MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
in
754         ``GET /api/videos``. """
755         try:
756             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
757         except edge_auth.EdgeAuthError as e:
758             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
759
760         cfg = request.app.state.config
761         sec = getattr(cfg, "security", None)
762         if getattr(sec, "allowed_video_root", None):
763             raise HTTPException(
764                 status_code=403,
765                 detail=(
766                     "storing into a custom medium is disabled on this hosted deployment "
767                     "(security.allowed_video_root is set); upload to local instead."
768                 ),
769             )
770         if bool(req.uploaded_path) == bool(req.source_uri):
771             raise HTTPException(
772                 status_code=400,
773                 detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
774             )
775         if req.uploaded_path and not req.medium_uri:

```

```

776         raise HTTPException(
777             status_code=400, detail="'medium_uri' is required when uploading a file."
778         )
779
780     storage_settings = _storage_settings(cfg)
781
782     # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
783     if req.uploaded_path:
784         try:
785             validate_input_path(req.uploaded_path, allowed_root=None)
786             src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
787         except ValueError as e:
788             raise HTTPException(status_code=400, detail=str(e)) from e
789         if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
790             raise HTTPException(
791                 status_code=404,
792                 detail=f"uploaded file not found at '{req.uploaded_path}'",
793             )
794         local_working = src_ref.key
795     else:
796         try:
797             validate_input_path(req.source_uri, allowed_root=None)
798             src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
799         except ValueError as e:
800             raise HTTPException(status_code=400, detail=str(e)) from e
801         # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
802         if src_ref.backend != "local":
803             need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
804             require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
805         try:
806             local_working = storage.acquire_local_input(
807                 req.source_uri, getattr(cfg, "storage", None)
808             )
809         except ValueError as e:
810             raise HTTPException(status_code=400, detail=str(e)) from e
811         except Exception as e: # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
812             raise HTTPException(
813                 status_code=502, detail=f"could not resolve source_uri: {e}"
814             ) from e
815         if not os.path.isfile(local_working):
816             raise HTTPException(
817                 status_code=404, detail="source_uri did not resolve to a readable file."
818             )
819
820     cleanup_raw = req.source_uri or ""
821     try:
822         video_id = compute_video_id(local_working)
823         ext = os.path.splitext(local_working)[1] or ".mp4"
824
825         # 2) Decide the durable location + copy only when needed.
826         copied = False
827         if req.source_uri:
828             stored_uri = (
829                 req.source_uri
830             ) # already at rest in its medium ? no copy (D14b)
831             medium_backend = src_ref.backend
832         else:
833             try:
834                 medium_ref = storage.parse_uri(req.medium_uri)
835             except ValueError as e:
836                 raise HTTPException(
837                     status_code=400, detail=f"invalid medium_uri: {e}"

```

```

838         ) from e
839         medium_backend = medium_ref.backend
840         if medium_ref.backend == "local":
841             # "This computer" ? the upload already lives here; it IS the durable
copy (no duplicate).
842             stored_uri = os.path.abspath(local_working)
843         else:
844             dest = _asset_dest_uri(req.medium_uri, video_id, ext)
845             # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud
bucket is warn-only.
846             local_dir = _medium_local_dir(medium_ref, storage_settings)
847             if local_dir is not None:
848                 require_free_bytes(
849                     local_dir, os.path.getsize(local_working), stage="store"
850                 )
851             else:
852                 logger.info(
853                     "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota
warn only).",
854                     medium_ref.backend,
855                 )
856             try:
857                 stored_uri = storage.put(
858                     local_working, dest, config=storage_settings
859                 )
860                 copied = True
861             except ValueError as e:
862                 raise HTTPException(status_code=400, detail=str(e)) from e
863             except Exception as e: # noqa: BLE001 ? a put failure is surfaced,
never silently dropped
864                 logger.error(
865                     "[ASSETS] put failed video_id=%s dest=%s: %s",
866                     video_id,
867                     dest,
868                     e,
869                     exc_info=True,
870                 )
871                 raise HTTPException(
872                     status_code=502, detail=f"could not store into the medium: {e}"
873                 ) from e
874
875         # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
videos").
876         if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
877             raise HTTPException(status_code=500, detail="failed to register the asset.")
878         logger.info(
879             "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
880             video_id,
881             medium_backend,
882             stored_uri,
883             copied,
884             owner_id,
885         )
886         return JSONResponse(
887             status_code=201,
888             content={
889                 "video_id": video_id,
890                 "medium": medium_backend,
891                 "stored_uri": stored_uri,
892                 "copied": copied,
893                 "sport": req.sport,
894                 "status": "stored",
895             },
896         )
897     finally:

```

```

898         if cleanup_raw:
899             storage.cleanup_acquired_local_input(
900                 cleanup_raw, local_working, getattr(cfg, "storage", None)
901             )
902
903
904     @app.post("/api/process", status_code=202)
905     def process_video(req: ProcessRequest, request: Request):
906         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
907
908         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
909         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
910         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
911         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
912         """
913         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
914         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
915         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
916         try:
917             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
918         except edge_auth.EdgeAuthError as e:
919             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
920
921         allowed_root = getattr(
922             getattr(request.app.state.config, "security", None), "allowed_video_root", None
923         )
924         # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
925         # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
926         allowed_uri_root = getattr(
927             getattr(request.app.state.config, "storage", None), "input_root", None
928         )
929         try:
930             validate_input_path(
931                 req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
932             )
933             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
934             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
935             input_ref = storage.resolve_input_ref(
936                 req.video_path, getattr(request.app.state.config, "storage", None)
937             )
938         except ValueError as e:
939             raise HTTPException(status_code=400, detail=str(e)) from e
940         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
941         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
942         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
943         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
944         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
945         # rejects a non-local URI as out-of-root.
946         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
947             raise HTTPException(
948                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
949             )
950         # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER

```



```

951         # the cache probe (#492 review): a cached gs:// result never fetches to local
scratch, so
952         # low temp space must not 507 it; a cache MISS still fails fast pre-claim exactly as
before.
953         if req.segmenter and not segmenter_registry.get(req.segmenter):
954             available = list(segmenter_registry.list_available().keys())
955             raise HTTPException(
956                 status_code=400,
957                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958             )
959
960         # ATOMIC idempotent submit (#448) + submit-time md5 cache (#484): one hook claims-
or-dedups,
961         # and completes the job INSTANTLY when this video's detection already exists in the
DB or the
962         # bucket (no drain, no GPU dispatch). `force` opts out. Logic in backend.api.jobs.
963         job_id = uuid.uuid4().hex
964         outcome = jobs_api.submit_process_job(
965             request.app.state.db, request.app.state.config, job_id, req, owner_id, input_ref
966         )
967         if outcome is None:
968             raise HTTPException(status_code=500, detail="Failed to persist job record.")
969         queued, response = outcome
970         if queued:
971             _get_executor().submit(
972                 _drain_due_jobs, request.app.state.config, request.app.state.db
973             )
974         return response
975
976
977     @app.get("/api/jobs/{job_id}/cost")
978     def get_job_cost(job_id: str, request: Request):
979         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
980         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
981         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
982         cost = request.app.state.db.get_job_cost(job_id)
983         if cost is None:
984             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
985         fx = request.app.state.config.billing.fx_inr_per_usd
986         usd = cost.get("cost_usd")
987         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
988         cost["fx_inr_per_usd"] = fx
989         return cost
990
991
992     @app.get("/api/videos/{video_id}/cost")
993     def get_video_cost(video_id: str, request: Request):
994         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
995         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
996         agg = request.app.state.db.get_video_cost(video_id)
997         fx = request.app.state.config.billing.fx_inr_per_usd
998         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
999         agg["fx_inr_per_usd"] = fx
1000         return agg
1001
1002
1003     @app.get("/api/jobs/{job_id}")
1004     def get_job(job_id: str, request: Request):
1005         """Job state: {status, progress, result, reports}. `status` = execution state
(queued|running|done|failed); the pipeline verdict is result["status"]."""
1006

```

```

1007     job = request.app.state.db.get_job(job_id)
1008     if not job:
1009         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1010     result = job.get("result")
1011     return {
1012         "job_id": job["id"],
1013         "status": job["status"],
1014         "progress": job.get("progress"),
1015         "video_id": job.get("video_id"),
1016         "error": job.get("error"),
1017         "result": result,
1018         "reports": (result or {}).get("reports"),
1019         "created_at": job.get("created_at"),
1020         "started_at": job.get("started_at"),
1021         "finished_at": job.get("finished_at"),
1022     }
1023
1024
1025     app.include_router(jobs_api.router) # GET /api/jobs ? the processing-queue view (#448)
1026
1027
1028     # --- Chunked upload (B2, PR-2) -----
1029     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
1030     # in
1031     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
1032     # via
1033     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1034     # sequential-part logic, and abort cleanup ? is unchanged.
1035     from backend.api import uploads as uploads_api
1036
1037     app.include_router(uploads_api.router)
1038
1039
1040     # --- Highlight download (B3, PR-2) -----
1041
1042     @app.get("/api/highlights/{video_id}/{filename}")
1043     def download_highlight(video_id: str, filename: str, request: Request):
1044         """Stream a compiled highlight reel ? `filename` is the bare name given to
1045         /api/compile
1046         (which only writes into output_dir; the browser previously got back an unreachable
1047         server-local path)."""
1048         try:
1049             name = safe_output_filename(filename)
1050         except ValueError as e:
1051             raise HTTPException(status_code=400, detail=str(e)) from e
1052         if not request.app.state.db.get_video(video_id):
1053             raise HTTPException(status_code=404, detail="Indexed video record not found.")
1054         # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
1055         # URL (bytes flow
1056         # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
1057         # /tmp). The
1058         # exists/sign logic (with the active storage config) lives in
1059         # orchestration.signed_reel_url to keep
1060         # this module lean; best-effort ? any error falls back to the local file below.
1061         try:
1062             signed = signed_reel_url(request.app.state.config, video_id, name)
1063             if signed:
1064                 return RedirectResponse(signed, status_code=307)
1065         except Exception as e: # noqa: BLE001 ? signing/exists hiccup ? serve the local
1066             # file instead
1067             logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
1068                             video_id, e)
1069             output_dir = (
1070                 getattr(

```

```

1064         getattr(request.app.state.config, "output", None), "output_dir", "output"
1065     )
1066     or "output"
1067 )
1068 path = os.path.join(output_dir, name)
1069 try:
1070     path = resolve_under_root(path, output_dir)
1071 except ValueError as e:
1072     raise HTTPException(status_code=400, detail=str(e)) from e
1073 if not os.path.isfile(path):
1074     raise HTTPException(
1075         status_code=404,
1076         detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1077     )
1078 media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1079 return FileResponse(path, media_type=media, filename=name)
1080
1081
1082 @app.post("/api/query")
1083 def query_play_segments(req: QueryRequest, request: Request):
1084     filters = {
1085         "server": req.server,
1086         "receiver": req.receiver,
1087         "duration_min": req.duration_min,
1088         "event_type": req.event_type,
1089     }
1090     segments = request.app.state.db.query_play_segments(req.video_id, filters)
1091     if not segments and read_path_rehydrate(
1092         request.app.state.db, request.app.state.config, req.video_id
1093     ): # #485: DB = cache, bucket = truth ? re-hydrate before reporting nothing
1094         segments = request.app.state.db.query_play_segments(req.video_id, filters)
1095     return {"play_segments": segments}
1096
1097
1098 @app.post("/api/compile", status_code=202)
1099 def compile_highlights(req: CompileRequest, request: Request):
1100     """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
1101     reel (#329).
1102     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
1103     concat + watermark)
1104     can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
1105     synchronously returned a
1106     Cloudflare 524 while the engine kept going (the reel built, but the client saw
1107     "failed" and never
1108     got the URL). The cheap checks (video exists, segments match + survive the floor,
1109     safe filename)
1110     fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
1111     `result` carries
1112     {filepath, download_url} ? the same shape the sync endpoint returned, now under
1113     /api/jobs/{id}."""
1114     # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
1115     before queuing.
1116     try:
1117         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1118     except edge_auth.EdgeAuthError as e:
1119         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1120     try:
1121         video, _kept, out_name = _resolve_compile(
1122             request.app.state.db,
1123             request.app.state.config,
1124             req.video_id,
1125             req.segment_ids,
1126             req.output_filename,
1127         )

```

```

1121         except _CompileError as e:
1122             raise HTTPException(status_code=e.status_code, detail=e.detail) from e
1123
1124         job_id = uuid.uuid4().hex
1125         # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
1126         # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
1127         if not request.app.state.db.create_compile_job(
1128             job_id,
1129             req.video_id,
1130             video["filepath"],
1131             req.segment_ids,
1132             out_name,
1133             locale=req.locale,
1134             owner_id=owner_id,
1135         ):
1136             raise HTTPException(
1137                 status_code=500, detail="Failed to persist compile job record."
1138             )
1139         _get_executor().submit(
1140             _drain_due_jobs, request.app.state.config, request.app.state.db
1141         )

```

18. user (2026-07-08T05:46:18.587Z)

```

80: def submit_process_job(
81-     db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82-) -> Optional[Tuple[bool, JSONResponse]]:
83-     """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in one
hook.
84-
85-     Order matters (#492 review):
86-     1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side effects.
87-     2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
88-     fetches to local scratch, so low temp space must not reject it (review blocker 2).
89-     3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
90-     re-hydrate side effect, so the first request owns the cache completion (blocker 1).
91-     4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself serialized
92-     per video_id) and completes the job instantly: no drain, no GPU dispatch.
93-
94-     Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain; ``None`` ?
95-     DB failure (the caller 500s)."""
96-     from backend import orchestration # call-time seam: tests patch orchestration attrs
97-
98-     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99-     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100-         db, cfg, video_id
101-     )
102-     if input_ref.backend != "local" and not probed:
103-         _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
104-         outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
105-         if outcome is None:
106-             return None
107-         created, dedup_response = outcome
108-         if not created:
109-             assert dedup_response is not None # claim contract: not-created ? dedup response
110-             return (False, dedup_response)
111-         if probed:
112-             cached = orchestration.cached_process_result(
113-                 db, cfg, video_id, req.video_path, requested_compute=req.compute
114-             )
115-             if cached is not None:
116-                 db.mark_job_done(job_id, cached)
117-             return (

```

```

118-         False,
119-         JSONResponse(
120-             status_code=202,
121-             content={
122-                 "job_id": job_id,
123-                 "status": "done",
124-                 "poll": f"/api/jobs/{job_id}",
125-                 "cached": True,
126-             },
127-         ),
128-     )
129-     # The probe said hit but realizing it failed (report vanished \ corrupt csv): fall
130-     # back to the normal queued dispatch ? but the scratch guard was skipped above, so
131-     # must run NOW. If it fails, the already-claimed row is marked failed BEFORE raising
132-     # (a bare raise would leave a phantom 'queued' job for the drain to grab).
133-     if input_ref.backend != "local":
134-         try:
135-             _fetch_scratch_guard(cfg, input_ref)
136-         except HTTPException:
137-             db.mark_job_failed(
138-                 job_id,
139-                 "cache re-hydrate failed and local scratch cannot hold a fetch of "
140-                 "the source; free disk space or retry later",
141-             )
142-             raise
143-     return (
144-         True,
145-         JSONResponse(
146-             status_code=202,
147-             content={
148-                 "job_id": job_id,
149-                 "status": "queued",
150-                 "poll": f"/api/jobs/{job_id}",
151-             },
152-         ),
153-     )
154-
155-# Execution states a job can be in. ACTIVE = still doing (or about to do) work; the queue
156-# view defaults
157-# to these. done/failed are terminal and only surface under ?status=all (or an explicit
158-# single status).
159-ACTIVE_JOB_STATES = ("queued", "running", "waiting")
160-
161-# LISTABLE_JOB_STATES = frozenset(ACTIVE_JOB_STATES) | {"done", "failed"}
162-
163-@router.get("/api/jobs")
164-def list_jobs(request: Request, status: str = "active"):
165-    """The caller's jobs (tenant-scoped), newest first. ``status=active`` (default) ?
166-    queued|running|waiting; ``status=all`` ? recent jobs of any state; a single explicit
167-    state is also
168-    accepted (anything unknown degrades to the active queue ? never a raw 500). Light
169-    projection: the
170-    heavy per-job ``play_segments`` payload is dropped; ``result_status`` carries the
171-    terminal verdict."""
172-    try:
173-        owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
174-    except edge_auth.EdgeAuthError as e:
175-        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
176-    if status == "all":
177-        statuses: Optional[List[str]] = None
178-    elif status in _LISTABLE_JOB_STATES:
179-        statuses = [status]
180-    else: # "active" and any unknown value ? the active queue
181-        statuses = list(ACTIVE_JOB_STATES)

```

```

177- rows = request.app.state.db.list_jobs(owner_id, statuses=statuses, limit=50)
178- return {
179-     "jobs": [
180-         {
181-             "job_id": j["id"],
182-             "kind": j.get("kind") or "process",
183-             "status": j["status"],
184-             "progress": j.get("progress"),
185-             "video_id": j.get("video_id"),
186-             "source": j.get("video_path"),
187-             "compute": j.get("compute"),
188-             "result_status": j.get("result_status"),
189-             "error": j.get("error"),
190-             "created_at": j.get("created_at"),
191-             "started_at": j.get("started_at"),
192-             "finished_at": j.get("finished_at"),
193-         }
194-         for j in rows
195-     ]
196- }

```

19. user (2026-07-08T05:46:19.615Z)

```

15: ``input_root`` plus the streamed md5 as ``video_id``.
27:import hashlib
43:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
185:         "hasher": hashlib.md5(),
254:         st["hasher"].update(data)
288:         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
300:         # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301:         # part IS compute_video_id's digest (same bytes, same algorithm), so /api/process
316:         "video_id": st["hasher"].hexdigest(),
329:     video_id = compute_video_id(final_path)
332:     "video_id": video_id,

```

20. user (2026-07-08T05:46:25.417Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2). Where they assemble depends
5     on the input backend (CONTROL_PLANE_DURABLE_REBUILD P2):
6
7     * ``storage.input_backend == "local"`` (appliance/default): parts append to a partial
8     file under security.upload_dir, exactly as before.
9     * a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10     through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
11     while an incremental MD5 accumulates ? the assembled file never touches local
12     disk/tmpfs. The 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload
13     (#480/#471); after this, instance memory bounds a CHUNK, not the video. ``complete``
14     returns the staged URI (inside the #465 input jail by construction ? it is under
15     ``input_root``) plus the streamed md5 as ``video_id``.
16
17     Upload state is in-process by design (this is the single-box alpha): a server restart
18     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
19     arrives with PR-3 identity scoping.
20
21     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
22     ``upload_cfg`` resolves it through the ``backend.main`` module object so it always
23     reads the live value the server set at startup (and so tests that monkeypatch
24     ``backend.main.global_config`` flow through unchanged).
25     """
26
27     import hashlib

```

```

28 import os
29 import threading
30 import unicodedata
31 import uuid
32 from typing import Any, Dict, Optional
33
34 from fastapi import APIRouter, HTTPException, Request
35 from pydantic import BaseModel
36 from starlette.concurrency import run_in_threadpool
37
38 from backend import storage
39 from backend.utils.app_home import (
40     resolve_output_dir, # P0a: app-home-aware path resolution
41 )
42 from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
43 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
44 from backend.utils.paths import safe_output_filename
45
46 router = APIRouter()
47
48 _uploads: Dict[str, Dict[str, Any]] = {}
49 _uploads_lock = threading.Lock()
50
51
52 class UploadInitRequest(BaseModel):
53     filename: str
54     total_size_bytes: Optional[int] = None
55
56
57 class UploadCompleteRequest(BaseModel):
58     total_parts: int
59
60
61 def _upload_cfg():
62     # Resolved lazily (import inside the function) to read the LIVE startup config that
63     # main() sets on the module global, and to avoid a circular import at module load
64     # (main.py imports this router; this module must not import main at import time).
65     import backend.main as _main
66
67     sec = getattr(_main.global_config, "security", None)
68     upload_dir = resolve_output_dir(
69         getattr(sec, "upload_dir", None) or "output/uploads"
70     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
71     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
72     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
73     exts = [
74         e.lower().lstrip(".")
75         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
76     ]
77     return upload_dir, max_bytes, part_bytes, exts
78
79
80 def upload_caps(cfg) -> Dict[str, int]:
81     """The upload limits /api/capabilities advertises (#480 item 5, CP-rebuild P5a).
82
83     ``max_upload_mb`` is what the SPA's pre-flight guard rejects oversize files against
84     ?
85     without it the guard is dead code and the user learns from a mid-upload 413.
86     ``max_part_mb`` is informational: each /api/upload/init response carries the
87     authoritative value the client actually chunks by."""
88     sec = getattr(cfg, "security", None)
89     return {
90         "max_upload_mb": int(getattr(sec, "max_upload_mb", 4096) or 4096),
91         "max_part_mb": int(getattr(sec, "max_part_mb", 95) or 95),
92     }

```

```

92
93
94 def _staging_target() -> "Optional[Dict[str, Any]]":
95     """The remote staging root + storage settings, or None for the local-file flow.
96
97     Remote staging is selected by the same input config /api/process resolves against
98     (storage.input_backend + input_root), so a staged upload is submittable verbatim.
99     """
100     import backend.main as _main
101
102     sc = getattr(_main.global_config, "storage", None)
103     if sc is None:
104         return None
105     input_backend = getattr(sc, "input_backend", "local") or "local"
106     input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
107     if input_backend == "local" or not input_root:
108         return None
109     cfg = sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
110     return {"root": input_root, "config": cfg}
111
112
113 def _abort_upload(upload_id: str) -> None:
114     with _uploads_lock:
115         st = _uploads.pop(upload_id, None)
116         if not st:
117             return
118         writer = st.get("writer")
119         if writer is not None:
120             # A streaming writer has no cancel ? close() finalizes whatever was staged, then
121             # the partial object is deleted. Both are best-effort: the upload is already
122             # aborted from the client's point of view.
123             try:
124                 writer.close()
125             except Exception: # noqa: BLE001 ? abort cleanup must never mask the abort
126                 pass
127             try:
128                 ref = storage.parse_uri(st["stage_uri"])
129                 storage.get_storage(ref.backend, config=st.get("config")).delete(ref)
130             except Exception: # noqa: BLE001
131                 pass
132             return
133         try:
134             os.remove(st["tmp_path"])
135         except OSError:
136             pass
137
138
139 def _normalize_upload_filename(filename: str) -> str:
140     """Normalize browser-provided file names before extension validation.
141
142     Some mobile/file-provider pickers can hand us visually normal names with
143     compatibility characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
144     existing traversal/null-byte checks in ``safe_output_filename`` as the authority after this
145     cleanup.
146     """
147     return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
148
149 @router.post("/api/upload/init")
150 def upload_init(req: UploadInitRequest):
151     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
152     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
153     try:

```



```

154         name = safe_output_filename(_normalize_upload_filename(req.filename))
155     except ValueError as e:
156         raise HTTPException(status_code=400, detail=str(e)) from e
157     ext = os.path.splitext(name)[1].lower().lstrip(".")
158     if ext not in exts:
159         raise HTTPException(
160             status_code=400,
161             detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
162         )
163     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
164         raise HTTPException(
165             status_code=413,
166             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
167         )
168     upload_id = uuid.uuid4().hex
169     staging = _staging_target()
170     if staging is not None:
171         # Remote staging: stream parts straight into the bucket under the input root ?
172         # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
173         # upload_id in the key keeps concurrent same-name uploads distinct, and the
174         # space guard is skipped: no local bytes are consumed.
175         stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
176         try:
177             writer = storage.open_writer(stage_uri, config=staging["config"])
178         except NotImplementedError:
179             staging = None # backend can't stream (e.g. mounted-Drive) ? stage locally
180     else:
181         with _uploads_lock:
182             _uploads[upload_id] = {
183                 "filename": name,
184                 "writer": writer,
185                 "hasher": hashlib.md5(),
186                 "stage_uri": stage_uri,
187                 "config": staging["config"],
188                 "parts": 0,
189                 "bytes": 0,
190             }
191         return {
192             "upload_id": upload_id,
193             "max_part_mb": part_bytes // (1024 * 1024),
194             "next_part": f"/api/upload/{upload_id}/part/0",
195         }
196     os.makedirs(upload_dir, exist_ok=True)
197     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
198     # effort ? an
199     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
200     # (never blocks a
201     # legit upload); the per-part total cap in upload_part is the hard backstop either
202     # way.
203     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
204     tmp_path = os.path.join(upload_dir, f"partial-{upload_id}")
205     open(tmp_path, "wb").close()
206     with _uploads_lock:
207         _uploads[upload_id] = {
208             "filename": name,
209             "tmp_path": tmp_path,
210             "parts": 0,
211             "bytes": 0,
212         }
213     return {
214         "upload_id": upload_id,
215         "max_part_mb": part_bytes // (1024 * 1024),
216         "next_part": f"/api/upload/{upload_id}/part/0",
217     }

```

```

215
216
217 @router.put("/api/upload/{upload_id}/part/{index}")
218 async def upload_part(upload_id: str, index: int, request: Request):
219     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
220     _, max_bytes, part_bytes, _ = _upload_cfg()
221     with _uploads_lock:
222         st = _uploads.get(upload_id)
223         if st is None:
224             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
225         if index != st["parts"]:
226             raise HTTPException(
227                 status_code=409,
228                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
229             )
230
231         received = 0
232         overflow = None
233         if st.get("writer") is not None:
234             # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
an
235             # oversize stream), then hand it to the blocking bucket writer in one threadpool
236             # hop so a network flush never stalls the event loop mid-upload.
237             buf = bytearray()
238             async for chunk in request.stream():
239                 received += len(chunk)
240                 if received > part_bytes:
241                     overflow = (
242                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
243                     )
244                     break
245                 if st["bytes"] + received > max_bytes:
246                     overflow = (
247                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
248                     )
249                     break
250                 buf += chunk
251             if not overflow and buf:
252                 data = bytes(buf)
253                 await run_in_threadpool(st["writer"].write, data)
254                 st["hasher"].update(data)
255         else:
256             with open(st["tmp_path"], "ab") as fh:
257                 async for chunk in request.stream():
258                     received += len(chunk)
259                     if received > part_bytes:
260                         overflow = (
261                             f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
262                         )
263                         break
264                     if st["bytes"] + received > max_bytes:
265                         overflow = (
266                             f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
267                         )
268                         break
269                     fh.write(chunk)
270         if overflow:
271             _abort_upload(upload_id)
272             raise HTTPException(
273                 status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
274             )
275

```

```

276     with _uploads_lock:
277         st["parts"] += 1
278         st["bytes"] += received
279     return {
280         "upload_id": upload_id,
281         "received_parts": st["parts"],
282         "received_bytes": st["bytes"],
283     }
284
285
286     @router.post("/api/upload/{upload_id}/complete")
287     def upload_complete(upload_id: str, req: UploadCompleteRequest):
288         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
289         upload_dir, _, _, _ = _upload_cfg()
290         with _uploads_lock:
291             st = _uploads.get(upload_id)
292         if st is None:
293             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
294         if st["parts"] == 0 or st["parts"] != req.total_parts:
295             raise HTTPException(
296                 status_code=409,
297                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
298             )
299         if st.get("writer") is not None:
300             # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301             # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
302             # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
303             # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
304             try:
305                 st["writer"].close()
306             except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
307                 _abort_upload(upload_id)
308                 raise HTTPException(
309                     status_code=502,
310                     detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
311                 ) from e
312         with _uploads_lock:
313             _uploads.pop(upload_id, None)
314         return {
315             "path": st["stage_uri"],
316             "video_id": st["hasher"].hexdigest(),
317             "size_bytes": st["bytes"],
318             "next": "POST /api/process with this path",
319         }
320         final_path = os.path.join(upload_dir, st["filename"])
321         if os.path.exists(
322             final_path
323         ): # never overwrite an earlier upload with the same name
324             stem, ext = os.path.splitext(st["filename"])
325             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326         os.replace(st["tmp_path"], final_path)
327         with _uploads_lock:
328             _uploads.pop(upload_id, None)
329         video_id = compute_video_id(final_path)
330         return {
331             "path": os.path.abspath(final_path),
332             "video_id": video_id,
333             "size_bytes": os.path.getsize(final_path),
334             "next": "POST /api/process with this path",
335         }
336
337

```

```

338 @router.delete("/api/upload/{upload_id}")
339 def upload_abort(upload_id: str):
340     """Abort an in-flight upload and delete its partial file."""
341     with _uploads_lock:
342         known = upload_id in _uploads
343         if not known:
344             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
345         _abort_upload(upload_id)
346         return {"status": "aborted", "upload_id": upload_id}
347

```

21. user (2026-07-08T05:46:36.370Z)

```

258: class ProcessRequest(BaseModel):
259-     video_path: str
260-     player_pool: Optional[List[str]] = None
261-     max_frames: Optional[int] = None
262-     segmenter: Optional[str] = None
263-     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job for
PMF
264-     # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265-     locale: Optional[str] = None
266-     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267-     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268-     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269-     compute: Optional[str] = None
270-     # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271-     # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272-     force: bool = False
273-
274-
275: class AnnotationsRequest(BaseModel):
276-     # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8, from
the Studio
277-     # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written
verbatim.
278-     video_id: str
279-     csv: str
280-
281-
282: class AssetRequest(BaseModel):
283-     # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:

```

22. user (2026-07-08T05:46:37.503Z)

```

29: def claim_process_job(
30-     db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
31-) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
32-     """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
33-     ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
34-     double-click / retry ? can't both create a job and start a second PAID GPU run. Returns
35-     ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when False
the caller
36-     just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None`` on
DB failure.
37-     ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the

```

```

input isn't
38-     hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39-     the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
40-     can re-associate the job with its bucket outputs (CP-rebuild $4.2); "" ? the drain fills
it.""
41-     claimed = db.claim_or_create_process_job(
42-         job_id,
43-         req.video_path,
44-         segmenter=req.segmenter,
45-         player_pool=req.player_pool,
46-         max_frames=req.max_frames,
47-         owner_id=owner_id,
48-         locale=req.locale,
49-         compute=req.compute,
50-         force=req.force,
51-         video_id=video_id,
52-     )
53-     if claimed is None:
54-         return None
55-     if claimed["created"]:
56-         return (True, None)
57-     return (
58-         False,
59-         JSONResponse(
60-             status_code=202,
61-             content={
62-                 "job_id": claimed["job_id"],
63-                 "status": claimed["status"],
64-                 "poll": f"/api/jobs/{claimed['job_id']}",
65-                 "deduplicated": True,
66-             },
67-         ),
68-     )
69-

```

23. user (2026-07-08T05:46:49.219Z)

```

backend\config\loader.py:200:         ct = merged["deployment"].get("compute_target")
backend\config\loader.py:203:         # An active profile with no explicit compute_target
(e.g. config.json blanked it
backend\config\loader.py:205:         # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend\config\loader.py:206:         merged["deployment"]["compute_target"] = canonical
backend\config\loader.py:209:         "[CONFIG] deployment profile %r normally uses
compute_target=%r but "
backend\config\loader.py:216:         "[CONFIG] deployment profile ACTIVE: %s
(compute_target=%s)",
backend\config\loader.py:218:         merged["deployment"].get("compute_target"),
backend\config\loader.py:254:# local storage and the compute_target, so this stays the handful
of knobs a fresh, non-technical
backend\config\models.py:24:class ValidationConfig(BaseModel):
backend\config\models.py:413:class StorageConfig(BaseModel):
backend\config\models.py:423:     output_root: str = ""
backend\config\models.py:424:     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the
output `backend`/`output_root`).
backend\config\models.py:427:     # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
backend\config\models.py:428:     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is
empty, prefixed with ``input_backend``
backend\config\models.py:432:     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-
api"] = "local"
backend\config\models.py:433:     input_root: str = ""
backend\config\models.py:440:     # Durable workspace root URI. "" => precedence: output_root,

```

```

then output.output_dir.
backend\config\models.py:454:class DeploymentConfig(BaseModel):
backend\config\models.py:471:     compute_target: Literal["", "local_gpu", "cloud_run",
"cpu_mock"] = ""
backend\config\models.py:473:     # bucket-poll CONCURRENTLY when compute_target=="cloud_run".
Each is an I/O-bound Cloud Run Job
backend\config\models.py:477:     max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
backend\config\models.py:483:     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
backend\config\presets.py:30:# Each profile maps 1:1 to its canonical compute_target. Used to
apply the preset and to warn
backend\config\presets.py:31:# when config.json explicitly overrides compute_target into a state
inconsistent with the profile.
backend\config\presets.py:39:# that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
backend\config\presets.py:41:# these bundles ? ``input_backend`` (M2) and the configurable
output/workspace base (M3,
backend\config\presets.py:45:         "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:52:         "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:75:         "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},

```

24. user (2026-07-08T05:46:50.665Z)

No matches found

25. user (2026-07-08T05:46:58.500Z)

```

backend\infrastructure\database_media.py:17:     def add_video(
backend\infrastructure\database_media.py:42:     def set_validation_cache(
backend\infrastructure\database_media.py:70:     def get_validation_cache(self, video_id: str) ->
Optional[Dict[str, Any]]:
backend\infrastructure\database_media.py:97:     def segment_watermarks(self, video_id: str) ->
Tuple[int, int]:
backend\infrastructure\database_media.py:115:     def prune_segments(
backend\infrastructure\database_media.py:154:     def add_play_segment(
backend\infrastructure\database_jobs.py:120:     def set_job_video_id(self, job_id: str,
video_id: str) -> bool:
backend\infrastructure\database_jobs.py:127:     def mark_job_done(self, job_id: str, result:
Dict[str, Any]) -> bool:
backend\infrastructure\database_jobs.py:228:     def claim_or_create_process_job(
backend\infrastructure\database_jobs.py:508:     def claim_due_job(self) -> Optional[Dict[str,
Any]]:
backend\infrastructure\database_jobs.py:628:     def interrupted_remote_process_jobs(self) ->
List[Dict[str, Any]]:

```

26. user (2026-07-08T05:47:00.127Z)

<tool_use_error>Path does not exist: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\worker.py. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer.</tool_use_error>

27. user (2026-07-08T05:47:06.475Z)

```

backend\compute\cloud_run.py:264:         if spec.skip_validation:
backend\compute\cloud_run.py:267:             argv.append("--skip-validation")
backend\compute\base.py:51:     skip_validation: bool = False
backend\worker\__main__.py:194:         skip_validation=bool(getattr(args, "skip_validation",
False)),
backend\worker\__main__.py:280:     # --skip-validation (set by the dispatcher in
orchestration._maybe_dispatch_remote) skips it; the
backend\worker\__main__.py:282:     if getattr(args, "skip_validation", False):
backend\worker\__main__.py:284:         "[worker] validation SKIPPED (--skip-validation): the

```

```

dispatcher is the authoritative "
backend\worker\_main.py:754:      "--skip-validation",
backend\orchestration.py:631:      skip_validation=True,

```

28. user (2026-07-08T05:47:11.740Z)

```

130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. ""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

```

```

178
179     A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180     from the local 2/3), never a raw traceback:
181     * rc 4 ? the bucket poll exhausted its budget (`PolicyTimeout`): the execution
may still be
182         running (the shim best-effort cancels it).
183     * rc 5 ? the execution has TERMINATED without a marker
(`RemoteExecutionFailed`): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186     from backend.compute import RemoteExecutionFailed
187     from backend.infrastructure.execution_policy import PolicyTimeout
188
189     spec = ComputeJobSpec(
190         video_uri=args.video_uri,
191         out_uri=args.out_uri,
192         segmenter=args.segmenter,
193         config_overrides=tuple(args.set or ()),
194         skip_validation=bool(getattr(args, "skip_validation", False)),
195     )
196     try:
197         result = compute.run_infer(spec)
198     except RemoteExecutionFailed as e:
199         logger.error(
200             "[worker] remote execution terminated without a completion marker: %s", e
201         )
202         return 5
203     except PolicyTimeout as e:
204         logger.warning(
205             "[worker] remote compute did not complete (no marker within budget): %s", e
206         )
207         return 4
208     logger.info(
209         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210         result.video_id,
211         result.returncode,
212         result.outputs_uri,
213     )
214     return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to `done_uri` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,
233                 "status": status,
234             }
235             local = os.path.join(scratch, "_done.json")
236             with open(local, "w", encoding="utf-8") as f:

```



```

237         json.dump(marker, f)
238         storage.put(local, done_uri, config=storage_cfg)
239         logger.info("[worker] wrote completion marker -> %s", done_uri)
240     except Exception as e: # never fail a good run because the marker push hiccuped
241         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242
243
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
246         _apply_overrides(config, args.set)
247         if not _run_startup_checks(config):
248             return 2
249
250         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )

```

```

287         val_results, val_context = [], {}
288     else:
289         val_results, val_context = validator_registry.run_all_with_context(
290             local_video, config
291         )
292     failures = [r for r in val_results if not r.passed]
293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,
296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
309         # expires.
310         try:
311             _serialize_and_push_outputs(
312                 scratch,
313                 args.out_uri,
314                 video_id,
315                 [],
316                 "failed",
317                 storage_cfg,
318                 str(getattr(args, "video_uri", "") or ""),
319             )
320         except Exception as e: # never mask the real failure on an artifact-push hiccup
321             logger.warning("[worker] could not push failure artifacts: %s", e)
322         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
323         if getattr(args, "done_uri", None):
324             _write_done_marker(
325                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
326             )
327         return rc
328
329     segments: List[dict] = []
330     segmenter_actual = None
331     segmentation_ran = False
332     status = "failed"
333     try:
334         if failures:
335             print(
336                 "[worker] validation FAILED: "
337                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
338             )
339             return _fail(2)
340         seg_name = args.segmenter or config.indexing.default_segmenter
341         segmenter_cls = segmenter_registry.get(seg_name)
342         if not segmenter_cls:
343             print(
344                 f"[worker] unknown segmenter: {seg_name!r} "
345                 f"(have {list(segmenter_registry.list_available())})"

```

```

345         )
346         return _fail(2)
347     segmenter_actual = seg_name
348     result = segmenter_cls(db, config).process_video(
349         video_id, local_video, max_frames=args.max_frames
350     )
351     segmentation_ran = True
352     if isinstance(result, Failure):
353         print(f"[worker] segmenter FAILED: {result.message}")
354         return _fail(3)
355     segments = result.value if hasattr(result, "value") else result
356     status = "success"
357     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360     if not segments:
361         detector_impl = (
362             os.environ.get("RALLY_DETECTOR_IMPL")
363             or getattr(config.indexing, "detector_impl", None)
364             or "auto"
365         )
366         logger.warning(
367             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373             "detections, or the input resolution differs from the tuned proxy
resolution. "
374             "Re-run with -v for the native command + frame counts.",
375             video_id,
376             segmenter_actual,
377             detector_impl,
378         )
379     finally:
380         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381         emit_indexer_report(
382             db=db,
383             video_id=video_id,
384             video_path=local_video,
385             config=config,
386             val_results=val_results,
387             val_context=val_context,
388             segmenter_requested=args.segmenter,
389             segmenter_actual=segmenter_actual,

```

29. user (2026-07-08T05:47:17.904Z)

```

389         segmenter_actual=segmenter_actual,
390         was_reingest=False,
391         segmentation_ran=segmentation_ran,
392     )
393
394     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395     pushed = _serialize_and_push_outputs(

```

```

396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(
406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
415     # if getattr(args, "done_uri", None):
416     #     _write_done_marker(
417     #         args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418     #     )
419     return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
424     _VIDEO_EXTS = ("mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
434         filename.
435
436         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
437         prefix, so a
438         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
439
440         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
441         stamped)
442         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
443         stem)
444         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
445         the name, kept)
446         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
447         <name> survive)
448
449         Without the date strip, ``--trajectory-source detector`` can't match a dated label
450         to its bare
451         video stem, so every clip is skipped and train aborts with ``<2 videos``

```

```

(DATA_IN_GCS $7b #1).
445     """
446     name = fname.replace(".rallies.csv", "").replace(".csv", "")
447     return _LABEL_DATE_PREFIX.sub("", name)
448
449
450     def _probe_video_fps_width(path: str):
451         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
452
453         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458         rather than guesses.
459         """
460         import json
461         import subprocess
462
463         try:
464             out = subprocess.check_output(
465                 [
466                     ffprobe_bin(),
467                     "-v",
468                     "error",

```

30. user (2026-07-08T05:47:27.639Z)

```

17     def add_video(
18         self,
19         video_id: str,
20         filepath: str,
21         status: str,
22         validation_results: List[Dict[str, Any]],
23     ) -> bool:
24         """Register or update a video row WITHOUT touching its child segments.
25
26         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
27         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
28         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
29         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
30         row in place; clearing segments is now an explicit, deliberate operation
31         (clear_video_data / prune_segments).
32         """
33         return self._execute_write(
34             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
35             "ON CONFLICT(id) DO UPDATE SET "
36             "filepath=excluded.filepath, status=excluded.status, "
37             "validation_results=excluded.validation_results",
38             (video_id, filepath, status, json.dumps(validation_results)),
39             "Failed to add video",
40         )
41
42     def set_validation_cache(
43         self,
44         video_id: str,

```

```

45         fingerprint: str,
46         passed: bool,
47         validation_results: List[Dict[str, Any]],
48     ) -> bool:
49         """Cache a video's validation verdict keyed by its content id (the MD5
50         ``video_id``), so an
51         identical-bytes REUPLOAD can skip re-running the (minutes-long on heavy 4K)
52         validator suite.
53         ``fingerprint`` is ``ValidationRegistry.config_fingerprint(config)`` ? a hash of
54         the
55         validation config + sport + validator set + a schema version. A reader
56         (``get_validation_cache``)
57         only reuses the verdict while that fingerprint still matches the CURRENT config,
58         so a config
59         change re-validates instead of serving a stale verdict. ``passed`` is the
60         overall verdict and
61         ``validation_results`` the full ValidationResult list (dumped) so a cached FAIL
62         can replay the
63         same message and a cached PASS carries the same results into the
64         report/response. UPSERT on the
65         ``video_id`` PK keeps ONE (latest) verdict per video: a re-validation under a
66         NEW config
67         overwrites the stale row rather than accumulating history."""
68         return self._execute_write(
69             "INSERT INTO validation_cache (video_id, fingerprint, passed, results) "
70             "VALUES (?, ?, ?, ?) "
71             "ON CONFLICT(video_id) DO UPDATE SET "
72             "fingerprint=excluded.fingerprint, passed=excluded.passed, "
73             "results=excluded.results, created_at=CURRENT_TIMESTAMP",
74             (video_id, fingerprint, 1 if passed else 0, json.dumps(validation_results)),
75             "Failed to write validation cache",
76         )
77
78     def get_validation_cache(self, video_id: str) -> Optional[Dict[str, Any]]:
79         """The cached validation verdict for ``video_id`` ? ``{fingerprint, passed,
80         results}`` ? or
81         ``None`` when nothing is cached.
82
83         The caller compares ``fingerprint`` against the CURRENT
84         ``ValidationRegistry.config_fingerprint(config)`` and reuses the verdict ONLY on
85         a match (a
86         config / sport / validator-set change flips the fingerprint ? re-validate).
87         ``results`` is the
88         stored ValidationResult list (parsed from JSON), replayed so a cached PASS
89         proceeds to
90         segmentation and a cached FAIL returns the same failure ? in both cases without
91         re-decoding the
92         video."""
93         with self._connect() as conn:
94             row = (
95                 conn.cursor()
96                 .execute(
97                     "SELECT fingerprint, passed, results FROM validation_cache "
98                     "WHERE video_id = ?",
99                     (video_id,),
100                 )
101                 .fetchone()
102             )
103             if not row:
104                 return None
105             d = dict(row)
106             d["passed"] = bool(d["passed"])
107             d["results"] = json.loads(d["results"] or "[]")
108             return d

```

```

96
97     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
98         """Return (max play_segment id, max candidate_segment id) for a video (0 if
99         none).
100
101         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
102         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
103         """
104         with self._connect() as conn:
105             cur = conn.cursor()
106             p = cur.execute(
107                 "SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id = ?",
108                 (video_id,),
109             ).fetchone()[0]
110             c = cur.execute(
111                 "SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE video_id =
112                 ?",
113                 (video_id,),
114             ).fetchone()[0]
115             return int(p), int(c)
116
117     def prune_segments(
118         self,
119         video_id: str,
120         *,
121         play_upto: Optional[int] = None,
122         play_after: Optional[int] = None,
123         cand_upto: Optional[int] = None,
124         cand_after: Optional[int] = None,
125     ) -> None:
126         """Delete play/candidate segments by id range (events cascade from
127         play_segments).
128
129         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
130         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
131         rows
132         and keep the OLD ones when the run failed/produced nothing (`*_after`).
133         """
134         with self._connect() as conn:
135             cur = conn.cursor()
136             if play_upto is not None:
137                 cur.execute(
138                     "DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
139                     (video_id, play_upto),
140                 )
141             if play_after is not None:
142                 cur.execute(
143                     "DELETE FROM play_segments WHERE video_id = ? AND id > ?",
144                     (video_id, play_after),
145                 )
146             if cand_upto is not None:
147                 cur.execute(
148                     "DELETE FROM candidate_segments WHERE video_id = ? AND id <= ?",
149                     (video_id, cand_upto),
150                 )
151             if cand_after is not None:
152                 cur.execute(
153                     "DELETE FROM candidate_segments WHERE video_id = ? AND id > ?",
154                     (video_id, cand_after),
155                 )
156             conn.commit()
157
158     def add_play_segment(
159         self,
160         video_id: str,

```

```

157         start_time: float,
158         end_time: float,
159         server: Optional[str] = None,
160         receiver: Optional[str] = None,
161         metadata: Optional[Dict[str, Any]] = None,
162     ) -> Optional[int]:
163         try:
164             with self._connect() as conn:
165                 cursor = conn.cursor()
166                 cursor.execute(
167                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
168                     (
169                         video_id,
170                         start_time,
171                         end_time,
172                         end_time - start_time,
173                         server,
174                         receiver,
175                         json.dumps(metadata or {}),
176                     ),
177                 )
178                 conn.commit()
179                 return cursor.lastrowid
180         except Exception as e:
181             logger.error(f"Failed to add play segment: {e}")
182             return None
183
184     def add_event(
185         self,
186         segment_id: int,
187         timestamp: float,
188         event_type: str,
189         player: Optional[str] = None,
190         details: Optional[Dict[str, Any]] = None,
191     ) -> bool:
192         return self._execute_write(
193             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
194             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
195             "Failed to add event",
196         )

```

31. user (2026-07-08T05:47:28.981Z)

```

24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."
39             ),
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )

```



```

44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63         # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).

```

32. user (2026-07-08T05:47:36.804Z)

```

228     def claim_or_create_process_job(
229         self,
230         job_id: str,
231         video_path: str,
232         segmenter: Optional[str] = None,
233         player_pool: Optional[List[str]] = None,
234         max_frames: Optional[int] = None,
235         policy: Optional[Dict[str, Any]] = None,
236         owner_id: Optional[str] = None,
237         locale: Optional[str] = None,
238         compute: Optional[str] = None,
239         force: bool = False,
240         video_id: str = "",
241     ) -> Optional[Dict[str, Any]]:
242         """ATOMIC idempotent submit for a PROCESS job (fixes the read-before-insert race
? PR #481
243         review): in ONE write-locked statement, insert ``job_id`` as 'queued' ONLY IF no
active
244         (queued|running|waiting) process job already exists for (owner_id, video_path).
Two concurrent
245         identical submits (a double-click / retry) therefore can't both insert ? SQLite
serializes
246         writers, and the second's ``INSERT ? WHERE NOT EXISTS`` sees the first's row
(same
247         compare-and-set idiom as ``claim_due_job``, no read-then-write gap). ``force``
skips the guard
248         (deliberate reprocess ? always inserts). Returns
``{"job_id","status","created"}`` ? ``created``
249         False means an active job already existed and the caller must NOT dispatch a
second run ? or
250         ``None`` on failure.
251
252         ``video_id`` is the submit-time MD5 when the caller could resolve one cheaply
(#484 ?
253         gs:// object metadata); "" ? unknown-yet (the drain fills it after hashing, as
before).
254         Persisting it at CREATE is what lets a restarted control plane re-associate a
running
255         remote job with its bucket outputs (CONTROL_PLANE_DURABLE_REBUILD $4.2)."""
256         params = json.dumps({"force": True}) if force else None

```

```

257         vals = (
258             job_id,
259             video_path,
260             video_id,
261             segmenter,
262             json.dumps(player_pool) if player_pool is not None else None,
263             max_frames,
264             json.dumps(policy) if policy is not None else None,
265             owner_id,
266             locale,
267             compute,
268             params,
269         )
270         owner_clause = "owner_id IS NULL" if owner_id is None else "owner_id = ?"
271         guard = () if owner_id is None else (owner_id,)
272         cols = (
273             "id, video_path, video_id, segmenter, player_pool, max_frames, status,
policy, "
274             "owner_id, locale, compute, params"
275         )
276         try:
277             with self._connect() as conn:
278                 cur = conn.cursor()
279                 if force:
280                     cur.execute(
281                         f"INSERT INTO jobs ({cols}) VALUES (?, ?, ?, ?, ?, ?, 'queued',
?, ?, ?, ?, ?)",
282                         vals,
283                     )
284                     conn.commit()
285                     return {"job_id": job_id, "status": "queued", "created": True}
286                     # Conditional insert ? one atomic statement: only if no active process
287                     # this (owner, input). SQLite serializes writers, so a racing second
288                     # submit sees the row.
289                     cur.execute(
290                         f"INSERT INTO jobs ({cols}) "
291                         "SELECT ?, ?, ?, ?, ?, ?, 'queued', ?, ?, ?, ?, ? WHERE NOT EXISTS
("
292                         "SELECT 1 FROM jobs WHERE " + owner_clause + " AND video_path = ? "
293                         "AND status IN ('queued', 'running', 'waiting') "
294                         "AND (kind IS NULL OR kind != 'compile'))",
295                         vals + guard + (video_path,),
296                     )
297                     conn.commit()
298                     if cur.rowcount == 1:
299                         return {"job_id": job_id, "status": "queued", "created": True}
300                         # Lost the race (or a pre-existing active job): return the winner, no
301                         # second dispatch.
302                         row = (
303                             cur.execute(
304                                 "SELECT id, status FROM jobs WHERE " + owner_clause + " AND
video_path = ? "
305                                 "AND status IN ('queued', 'running', 'waiting') "
306                                 "AND (kind IS NULL OR kind != 'compile') ORDER BY created_at
DESC LIMIT 1",
307                                 guard + (video_path,),
308                             ).fetchone()
309                         )
310                         if row:
311                             return {"job_id": str(row["id"]), "status": str(row["status"]),
"created": False}
312                             return None
313             except Exception as e:
314                 logger.error(f"Failed to claim-or-create process job {job_id}: {e}")

```

```

313         return None
314
315     # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
316     -----
317     def get_pricebook_rates(
318         self, version: Optional[str] = None
319     ) -> Dict[str, float]:
320         """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
321         GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''`.
322         An unknown version ? ``{}`` (so every cost computes to 0.0)."""
323         gt = gpu_type or ""
324         rates: Dict[str, float] = {}
325         with self._connect() as conn:
326             rows = (
327                 conn.cursor()
328                 .execute(
329                     "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE
version = ?",
330                     (version,),
331                 )
332                 .fetchall()
333             )
334             for r in rows:
335                 res, row_gt, price = (
336                     r["resource"],
337                     (r["gpu_type"] or ""),
338                     float(r["unit_price_usd"]),

```

33. user (2026-07-08T05:47:37.519Z)

```

413     class StorageConfig(BaseModel):
414         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
415         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
416         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
417         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
418         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
419         (boto3/google auth chains supply credentials). See ``backend/storage/``.
420
421         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
422         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
423         output_root: str = ""
424         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
425         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
426         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
427         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
428         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
429         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
430         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
431         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
432         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433         input_root: str = ""

```

```

434     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
435     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
436     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
437     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
438     # the legacy flat layout is used unchanged.
439     single_folder_per_video: bool = False
440     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441     workspace_root: str = ""
442     # Cloud namespace prefix under the root so per-video folders sit tidily beside
443     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444     workspace_prefix: str = "workspaces"
445     local: dict[str, Any] = Field(default_factory=dict)
446     s3: dict[str, Any] = Field(default_factory=dict)
447     gcs: dict[str, Any] = Field(default_factory=dict)
448     gdrive: dict[str, Any] = Field(default_factory=dict)
449     # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451     gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
460         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
461         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
462         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
463         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
464
465         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
466
467         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays

```

```

the
480         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner

```

34. user (2026-07-08T05:47:46.633Z)

No files found

35. user (2026-07-08T05:47:47.750Z)

```

1477:     requeued, failed = db_instance.recover_stale_jobs()
1484:     resume_interrupted_remote_jobs(global_config, db_instance)
1635:     configure_drain(global_config) # drain concurrency from config before the executor
exists (#466)

```

36. assistant (2026-07-08T05:49:30.439Z)

I have the full picture now. Let me compose the structured map.

37. user (2026-07-08T05:52:13.863Z)

Structured output provided successfully